

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY8. AI Agent 구현 실습 1

목 차

1. Agent memory		3
2. Long term memory		19
3. Research agent		36

1. Agent memory

방금 말한 걸 또 묻는 봇

지금까지 만든 에이전트는 대화를 기억하지 못합니다. "내 이름은 길동이야" 다음에 "내 이름 뭐랬지?"라고 물으면 "모르겠어요"라고 답하죠. 원인은 LLM 이 **stateless(무상태)** 라서입니다. 이번 장에서 단기 기억을 붙입니다.

1. 금붕어 같은 봇

지금까지의 에이전트로 이런 대화를 해 보면 문제가 드러납니다.

사용자: 내 이름은 민준이야.
봇: 네, 민준님 반갑습니다!
사용자: 내 이름 뭐랬지?
봇: 죄송하지만 성함을 알려주지 않으셨어요. ← 방금 들었는데 잊어버림!

방금 "민준"이라고 말했는데, 다음 질문에서 까먹습니다. 마치 금붕어처럼 3초 전을 기억 못 하죠. 이래선 상담봇으로 쓸 수 없습니다. "방금 말한 주문번호로 환불해줘"라고 하면 "어떤 주문번호요?"라고 되물을 테니까요.

2. 왜 기억을 못 하나

원인은 단순합니다. LLM API는 상태를 저장하지 않습니다(**stateless**). 매 호출은 완전히 백지에서 시작합니다.

호출 1: "내 이름은 민준이야" → "반갑습니다" (호출 끝, 다 잊음)
호출 2: "내 이름 뭐랬지?" → "모르겠어요" (백지에서 시작, 호출 1을 모름)

호출 2는 호출 1을 전혀 모릅니다. 각 호출이 독립적이거든요. 백엔드 개발자라면 익숙할 겁니다 — 세션이 없는 순수 함수(**stateless HTTP**) 와 같죠. 이전 요청을 기억하지 못합니다.

비유: 매번 기억이 지워지는 영화 주인공 같습니다(메멘토). 방금 만난 사람도 다음 순간 처음 보는 사람이 되죠. 그래서 매번 "저는 누구입니다"를 다시 말해 줘야 합니다.

ChatGPT 는 지금까지의 대화 전체를 매번 다시 전송합니다.

모델이 기억하는 게 아니라, 애플리케이션이 대화 기록을 들고 있다가 매 호출에 끼워 넣는 거죠. 그래서 기억하는 "것처럼" 보입니다.

4. 해법 — 대화를 누적해 주입

기억을 만들려면 우리가 직접 이전 대화를 매번 다시 실어 보내야 합니다.

- 기억 없음: 호출2 = ["내 이름 뭐랬지?"] → 모름
- 기억 있음: 호출2 = ["내 이름은 민준", "반갑습니다", "내 이름 뭐랬지?"] → "민준님!" (이전 대화를 누적해 함께 보냄)

직접 구현하면 "메시지 리스트를 매번 append 해서 보내는" 코드가 됩니다. 번거롭죠. LangGraph의 `InMemorySaver`가 이 누적·저장·재주입을 자동화해 줍니다. 우리는 그냥 `thread_id`만 넘기면 되죠.

핵심 정리

- 지금까지 에이전트는 대화를 기억 못 함(금붕어).
- 원인: LLM은 **stateless** — 매 호출이 백지에서 시작.
- ChatGPT도 LLM은 기억 못 하고, 앱이 대화를 매번 주입.
- 해법: 이전 대화를 누적해 주입 → `InMemorySaver`가 자동화.

LLM은 stateless — 맥락은 우리가 넣는다

LLM API는 상태를 저장하지 않습니다(**stateless**). 매 호출은 백지에서 시작하죠. "기억하는 것처럼" 보이는 챗봇도, 사실은 지금까지의 모든 대화를 매번 다시 전송하는 것입니다. 모델이 기억하는 게 아니라, 애플리케이션이 기록을 주입합니다.

1. stateless란?

stateless(무상태)는 "이전 호출의 상태를 기억하지 않는다"는 뜻입니다. 각 호출이 완전히 독립적이죠.

- 호출 1 → 처리 → 응답 (끝나면 다 잊음)
- 호출 2 → 처리 → 응답 (호출 1을 전혀 모름, 백지에서 시작)

백엔드 개발자에게 익숙한 개념입니다. HTTP 가 stateless 조 — 각 요청이 독립적이고, 서버는 이전 요청을 기억하지 않습니다. 그래서 세션·쿠키로 상태를 따로 관리하잖아요. LLM 도 똑같습니다.

2. 기억 없음 vs 기억 있음

같은 대화를 두 방식으로 비교해 봅시다.

기억 없음 — 매번 백지

호출1: ["내 이름은 민준"] → "반갑습니다"
호출2: ["내 이름 뭐랬지?"] ← 백지! → "모르겠어요" X

호출 2는 "내 이름 뭐랬지?"만 받습니다. 호출 1의 "민준"을 모르니, 답할 수 없죠.

기억 있음 — 기록을 매번 주입

- 호출1: ["내 이름은 민준"] → "반갑습니다"
- 호출2: ["내 이름은 민준", "반갑습니다", "내 이름 뭐랬지?"] → "민준님이요!" ✓ (이전 대화를 누적 주입)

호출 2에 이전 대화 전체를 함께 넣으면, 모델이 "민준"을 보고 답할 수 있습니다. 차이는 단 하나 — 이전 기록을 같이 보냈느냐입니다.

3. "기억"은 착시다

핵심 깨달음: 모델은 절대 기억하지 않습니다. 우리가 기억하는 것처럼 보이게 만드는 거죠.

모델 입장: 매 호출 백지 (stateless, 불변)
앱 입장: 대화 기록을 들고 있다가 매번 주입 → 모델이 "기억하는 것처럼" 보임

ChatGPT, Claude, 모든 챗봇이 이렇게 동작합니다. 모델 자체는 stateless이고, 애플리케이션 계층이 대화를 누적·관리합니다. "기억"은 모델의 능력이 아니라 앱의 설계인 셈이죠.

비유: 의사가 환자를 볼 때, 의사가 모든 환자를 외우는 게 아니라 차트(기록)를 봅니다. 환자가 올 때 마다 그 차트를 펴서 "지난번에 이런 말씀 하셨죠"라고 하죠. LLM의 "기억"도 이 차트(대화 기록)를 매번 펴 보는 것입니다.

4. 직접 구현하면?

기억을 직접 만들면 이런 코드가 됩니다.

```
# 수동 메모리 (직접 구현)
history = [] # 대화 기록을 담을 리스트

def chat(user_text):
    history.append({"role": "user", "content": user_text}) # 사용자 발화 추가
    response = llm.invoke(history) # 전체 기록을 보냄
    history.append({"role": "assistant", "content": response}) # 봇 응답 추가
    return response
```

`history`에 대화를 계속 쌓고, 매 호출에 전체를 보냅니다. 7강에서 ReAct 루프의 `history`를 누적한 것과 같은 원리죠. 작동은 하지만, 세션이 여러 개면(여러 사용자) 관리가 복잡해집니다. 그래서 LangGraph가 이걸 자동화합니다(03번).

5. 토큰 비용 주의

stateless를 이해하면 중요한 사실이 보입니다 — **대화가 길어질수록 매번 보내는 양이 늘어납니다.**

```
호출 1: 메시지 1개 보냄 → 토큰 적음
호출 10: 메시지 19개 보냄 → 토큰 많음 (이전 대화 다 포함!)
```

"기억"은 공짜가 아닙니다. 대화가 길어지면 매 호출에 전체 기록을 보내니, **토큰(비용)이 누적**됩니다. 그래서 긴 대화는 오래된 메시지를 잘라내거나 요약해야 하죠.

이게 stateless의 대가입니다.

핵심 정리

- LLM은 **stateless** — 매 호출이 백지, 이전을 모름(HTTP 처럼).
- "기억"은 **이전 대화를 매번 누적** 주입하는 것(앱의 설계).
- 모델은 절대 기억 안 함 — **앱 계층**이 대화를 관리.
- 직접 구현하면 history 리스트를 누적해 보냄(ReAct 처럼).
- 대화가 길수록 **토큰(비용) 누적** → 트리밍·요약 필요.

InMemorySaver 와 thread_id

InMemorySaver 는 대화 기록을 자동으로 저장·복원하는 **체크포인트(checkpointer)** 입니다.

thread_id 는 "어느 대화인가"를 구분하는 키죠. 같은 thread_id 면 기억이 이어지고, 다른 thread_id 면 완전히 새 대화로 격리됩니다.

1. 두 가지 부품

단기 기억은 두 부품으로 구성됩니다.

- **InMemorySaver (체크포인트)**: 대화 상태(메시지들)를 저장하는 엔진. 호출이 끝날 때마다 현재 대화를 스냅샷으로 저장하고, 다음 호출 때 자동 복원.
- **thread_id**: "어느 대화인가"를 구분하는 식별자. 같은 thread_id로 호출하면 이전 기록이 이어짐.

InMemorySaver(저장소) 가 thread_id별로 대화를 보관합니다.

- `thread_id="user-A"` → [민준, 반갑습니다, ...]
- `thread_id="user-B"` → [서윤, 안녕하세요, ...]

호출 시 thread_id로 해당 대화를 복원합니다.

- `agent.invoke(..., config={"thread_id": "user-A"})` → A 대화 복원
- `agent.invoke(..., config={"thread_id": "user-B"})` → B 대화 복원

2. 체크포인트가 하는 일

02번에서 직접 `history` 리스트를 관리하던 걸, 체크포인트가 자동으로 합니다.

직접 구현(02번):	체크포인트(InMemorySaver):
<code>history.append(...)</code>	매 호출 끝에 자동 저장
<code>llm.invoke(history)</code>	다음 호출 때 자동 복원
<code>history.append(...)</code>	우리는 신경 쓸 필요 없음!

`checkpointer=InMemorySaver()` 한 줄만 붙이면, 대화 누적·저장·복원이 전부 자동입니다. 우리가 메시지를 일일이 append 할 필요가 없죠. 8강의 `create_agent`가 ReAct 루프를 자동화한 것처럼, InMemorySaver가 메모리 관리를 자동화합니다.

3. thread_id = 세션 라우팅

thread_id는 "어느 사용자의 어느 대화를 이어갈지"를 결정합니다.

```
agent.invoke({"messages": [...]}, {"configurable": {"thread_id": "user-A"}})
# thread_id="user-A" → A의 대화 맥락을 이어 간다.
```

- 같은 thread_id → 그 대화의 이전 기록이 이어짐
- 다른 thread_id → 완전히 새 대화(격리)

이게 멀티 사용자 서비스의 핵심입니다. 사용자마다 다른 thread_id를 주면, 사용자별 대화가 안전하게 분리됩니다. A의 대화가 B에게 새어 나가지 않죠.

4. 백엔드 매핑

thread_id는 여러분 서비스의 세션ID/사용자ID와 1:1로 매핑하면 됩니다.

항목	의미	백엔드 매핑
checkpointer	상태 저장 엔진	세션 스토어(메모리/DB)
thread_id	대화 식별자	세션ID 또는 사용자ID
InMemorySaver	인메모리 체크포인트	프로세스 재시작 시 사라짐
(영속) DB 체크포인트	디스크 저장	Redis/Postgres(운영)

```
# 로그인 사용자
config = {"configurable": {"thread_id": f"user-{user_id}"}}

# 비로그인 채팅 (세션 토큰)
config = {"configurable": {"thread_id": session_token}}
```

"이 요청은 어떤 사용자의 어떤 대화인가"를 thread_id 하나로 라우팅합니다. 백엔드 세션 관리와 똑같죠.

5. InMemorySaver의 한계 — 인메모리

이름 그대로 **InMemorySaver**는 **메모리에 저장**합니다. 중요한 한계가 있습니다.

⚠ 프로세스가 재시작되면 모든 대화가 사라진다!

서버를 껐다 켜면 모든 대화 기록이 날아갑니다. 메모리에만 있었으니까요. 그래서:

- 학습·단일 프로세스 데모 → InMemorySaver (간단)
- 운영 서비스 → DB 기반 체크포인트 (영속)

운영에서는 `langgraph-checkpoint-sqlite`, Postgres, Redis 같은 **DB 체크포인트**로 교체합니다. 그러면 서버를 재시작해도 대화가 유지되죠. 이 강은 학습용이라 InMemorySaver를 쓰고, 영속 사용자 정보는 20강에서 다룹니다.

6. 단기 기억 vs 장기 기억 (예고)

- 단기 기억(이 강): 한 대화 세션 안의 맥락. 세션이 끝나면 사라짐(InMemorySaver는 프로세스 재시작 시 소멸).
- 장기 기억(20강): "어제 말한 것"까지 영구 저장. 세션을 넘어 유지.

InMemorySaver는 단기 기억입니다. "지금 이 대화"는 기억하지만, 서버 재시작이나 새 세션에서는 잊죠. 사용자 프로필(이름·선호)을 영구히 기억하려면 20강의 장기 기억이 필요합니다.

핵심 정리

- **InMemorySaver** = 대화를 자동 저장·복원하는 체크포인트(02 번 수동 관리의 자동화).
- **thread_id** = "어느 대화인가"를 구분하는 키(세션 ID/사용자 ID 와 매핑).
- 같은 thread_id 는 **이어지고**, 다른 thread_id 는 **격리**.
- InMemorySaver 는 **인메모리** — 재시작 시 사라짐(학습용).
- 운영은 **DB 체크포인트**(영속), 장기 기억은 다음 장에서.

따라하기 — 기억하는 상담봇 만들기

목표

`create_agent`에 `checkpointer=InMemorySaver()`를 붙이고, 호출할 때마다 `config`로 `thread_id`를 넘겨 멀티턴 대화를 구현합니다. 실제 파일은 `code/ch19_memory.py`입니다.

0. 실습 준비

8강 이후와 동일한 LangChain/LangGraph 환경입니다. 추가 설치는 없습니다. `InMemorySaver`는 이미 설치된 `langgraph`에 포함돼 있습니다.

```
conda activate agentic
python code/common.py # GOOGLE_API_KEY: True 확인
```

1. 메모리 불인 에이전트

`create_agent`에 `checkpointer=InMemorySaver()`를 추가합니다. 이 한 줄이 전부입니다.

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_chat
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver # 인메모리 체크포인트

def build_agent():
    """단기기억(InMemorySaver)을 불인 상담 에이전트를 생성한다."""
    llm = get_chat(temperature=0.3)
    agent = create_agent(
        llm,
        tools=[], # 이번 강은 '기억'에 집중 — 도구 없음
        system_prompt="너는 승승장구물의 친절하 CS 상담원이다. 한국어 존댓말로 답하라.",
        checkpointer=InMemorySaver(), # ← 이 한 줄로 기억 활성화!
    )
    return agent
```

이전부터 써 온 `create_agent`에 `checkpointer` 인자 하나만 추가됐습니다. 그러면 대화 상태(메시지들)가 매 호출 끝에 자동 저장/복원됩니다. 우리가 메시지를 누적할 필요가 없죠.

2. 대화 함수 — config로 thread_id 전달

호출할 때마다 `config`로 `thread_id`를 넘깁니다.

```
def chat(agent, text: str, config: dict):
    """한 번의 대화 턴. config의 thread_id로 같은 세션을 식별한다."""
    # config의 thread_id가 같으면 이전 대화가 자동으로 이어진다(직접 append 불필요)
    result = agent.invoke({"messages": [{"role": "user", "content": text}], config)
    return result, result["messages"]
```

핵심: `agent.invoke`의 두 번째 인자로 `config`를 넘깁니다. 이 config의 `thread_id`가 같으면, 이전 대화가 자동으로 이어지죠. 우리는 새 메시지 하나만 보내면, `InMemorySaver`가 이전 기록을 붙여 줍니다.

3. 멀티턴 대화 실행

같은 `thread_id`로 여러 번 호출하면 대화가 이어집니다.

```
agent = build_agent()
cfg = {"configurable": {"thread_id": "user-A"}}

# 1턴: 이름 알려주기
chat(agent, "내 이름은 민준이야.", cfg)

# 2턴: 같은 thread_id로 물어보기 → 기억함!
result, _ = chat(agent, "내 이름 뭐랬지?", cfg)
print(result["messages"][-1].text)
```

예상 출력

민준님이라고 하셨습니다! 무엇을 도와드릴까요?

01번의 "금붕어 붓"이 이제 기억합니다! 2턴에서 "내 이름 뭐랬지?"라고 물으니 "민준님"이라고 답하죠.

`checkpointer` 한 줄과 같은 `thread_id` 덕분입니다. 우리가 대화 기록을 직접 관리하지 않았는데도요.

4. .text vs .content 주의

답변을 꺼낼 때 `.content` 대신 `.text` 를 쓴 데 주목하세요.

```
result["messages"][-1].text      # ← 항상 문자열 (안전)
# result["messages"][-1].content # ← Gemini에서 리스트일 수 있음 (위험)
```

LangChain v1에서 AIMessage의 `.content`는 모델에 따라 문자열이 아니라 **content blocks(리스트)**로 올 수 있습니다(Gemini는 thought signature 등을 블록으로 반환). 항상 문자열을 안전하게 얻으려면 `.text`를 쓰세요. 특히 슬라이싱(`[ :40 ]`)이나 문자열 연결에서 `.content`가 리스트면 오류가 납니다. 작은 차이지만 실무에서 자주 만나는 함정이죠.

5. 무엇이 바뀌었나

8강부터 써 온 코드와 비교하면 차이는 딱 두 가지입니다.

```
# 1. checkpointer 추가
agent = create_agent(llm, tools=[], system_prompt=..., checkpointer=InMemorySaver())

# 2. invoke에 config(thread_id) 전달
agent.invoke({"messages": [...]}, {"configurable": {"thread_id": "user-A"}})
```

이 두 가지만으로 stateless 에이전트가 기억하는 에이전트가 됐습니다. 복잡한 메모리 관리 코드 없이, 프레임워크가 다 해 주죠. 이게 LangGraph의 편리함입니다.

핵심 정리

- `checkpointer=InMemorySaver()` 한 줄로 기억 활성화.
- `invoke`의 두 번째 인자로 **`config(thread_id)`** 전달.
- 같은 `thread_id`면 이전 대화가 **자동으로 이어짐**(직접 `append` 불필요).
- 답변은 `.content` 대신 **`.text`**(Gemini 블록 대응).
- 두 가지(`checkpointer` + `config`)만으로 기억 구현.

따라하기 — 같은 세션 vs 다른 세션

목표

thread_id가 다르면 같은 에이전트라도 기억이 완전히 분리됨을 확인합니다. 이게 멀티 사용자 서비스에서 대화를 안전하게 격리하는 핵심 메커니즘입니다.

1. 시나리오 설계

같은 에이전트에 두 개의 **thread_id**(A, B)로 대화해 봅니다.

```
agent = build_agent()
cfg_a = {"configurable": {"thread_id": "user-A"}} # 세션 A
cfg_b = {"configurable": {"thread_id": "user-B"}} # 세션 B
```

A에 정보를 알려주고, B에서 물어보고, 다시 A로 돌아가 확인합니다. A와 B가 격리되는지 보는 거죠.

2. 세션 A에 정보 입력

```
chat(agent, "내 이름은 민준이야.", cfg_a)
final, _ = chat(agent, "내가 좋아하는 카테고리는 전자기기야. 기억해.", cfg_a)
print("[세션 A] 봇:", final["messages"][-1].text)
```

예상 출력

```
[세션 A] 봇: 네, 민준님. 전자기기를 선호하신다고 기억해 두겠습니다. ← 이름(앞 턴)도 기억
```

세션 A에서 "민준"과 "전자기기"를 알려줬습니다. 봇이 둘 다 기억하죠(앞 턴의 이름까지).

3. 다른 세션 B에서 물어보기

이제 다른 **thread_id**(B)로 같은 정보를 물어봅니다.

```
final_b, _ = chat(agent, "내 이름이 뭐라고 했지?", cfg_b)
print("[세션 B] 봇:", final_b["messages"][-1].text)
```

예상 출력

[세션 B] 봇: 죄송하지만 아직 성함을 알려주지 않으셨어요. ← 세션이 달라 모름!

세션 B는 "민준"을 모릅니다. 같은 에이전트 객체인데도요! thread_id가 다르니, A의 기록과 완전히 격리됐기 때문이죠. B는 A에서 무슨 대화가 오갔는지 전혀 모릅니다.

4. 세션 A로 복귀

다시 thread_id A로 돌아가 확인합니다.

```
final_a, msgs_a = chat(agent, "내 이름이랑 좋아하는 카테고리 다시 말해줘.", cfg_a)
print("[세션 A 복귀] 봇:", final_a["messages"][-1].text)
```

예상 출력

[세션 A 복귀] 봇: 민준님이시고, 선호 카테고리는 전자기기입니다. ← 여전히 기억!

A로 돌아오니 여전히 기억합니다. B에 갔다 왔어도 A의 대화는 그대로 보존됐죠. 각 thread_id가 독립적인 대화를 유지하니까요.

5. 관찰 포인트 — 같은 에이전트, 격리된 기억

이 실험의 핵심은:

같은 에이전트 객체인데도, thread_id가 다르면 기억이 완전히 분리된다.

하나의 `agent` 객체가 thread_id별로 대화를 따로 보관합니다.

- `thread_id="user-A"` → `[민준, 전자기기, ...]` (A의 대화)
- `thread_id="user-B"` → `[]` (B의 대화, A와 무관)

에이전트는 하나지만, InMemorySaver가 thread_id 별로 대화를 **따로따로** 저장합니다. 그래서 A와 B가 섞이지 않죠.

6. 멀티 사용자 서비스의 핵심

이 격리가 멀티 사용자 서비스의 필수 요소입니다.

사용자 1000명이 동시에 채팅 → thread_id 1000개
→ 에이전트는 하나, 대화는 1000개로 격리
→ A 사용자의 정보가 B 사용자에게 절대 새어 나가지 않음

만약 격리가 안 되면, A 사용자에게 "당신 이름은 민준"이라고 했는데 B 사용자에게도 "당신 이름은 민준"이라고 답하는 **개인정보 유출 사고**가 납니다. thread_id 격리가 이를 막죠. 그래서 thread_id를 사용자별로 정확히 부여하는 게 보안상 매우 중요합니다.

7. 대화 기록 들여다보기

`chat` 이 돌려준 메시지 리스트로 누적된 대화를 볼 수 있습니다.

```
print("user-A 누적 메시지 수:", len(msgs_a)) # 턴이 쌓일수록 증가
print("마지막 메시지:", msgs_a[-1].text[:40], "...")
```

예상 출력

```
user-A 누적 메시지 수: 8
마지막 메시지: 민준님이시고, 선호 카테고리는 전자기기입니다 ...
```

턴이 진행될수록 메시지가 쌓입니다. 이게 02번에서 본 "기억 = 누적"의 실체죠. 그리고 이게 길어지면 토 큰 문제가 생겨, 다음 절에서 관리법을 배웁니다.

핵심 정리

- 같은 에이전트라도 thread_id 가 다르면 기억이 **완전 격리**.
- 세션 A 의 정보를 세션 B 는 **모름**(독립적 대화).
- InMemorySaver 가 thread_id 별로 대화를 **따로 저장**.
- 멀티 사용자 서비스에서 **개인정보 유출을 막는 핵심**.
- thread_id 를 **사용자별로 정확히** 부여해야 안전.

긴 대화 메시지 관리

대화가 길어지면 메시지가 무한히 쌓여 **토큰·비용·지연이 폭발**합니다. 두 해법이 있습니다

— (a) **트리밍**: 최근 N 개만 남기기, (b) **요약 압축**: 오래된 대화를 한 줄로 요약.

둘 다 "입력을 줄여 비용을 통제"하는 기법입니다.

1. 대화가 길어지면 생기는 문제

02번에서 배웠듯, 기억은 "이전 대화를 매번 누적 주입"하는 것입니다. 그래서 대화가 길어지면:

1턴: 메시지 1개 보냄
10턴: 메시지 19개 보냄
100턴: 메시지 199개 보냄! → 토큰 폭발, 비용·지연 증가

매 호출에 **전체 대화**를 보내니, 대화가 길수록 입력 토큰이 선형으로 증가합니다(7강 토큰 관리와 같은 문제). 100턴쯤 가면 인사 한마디에도 199개 메시지를 보내야 하죠. 비효율적이고 비쌉니다.

2. 두 가지 해법

이 문제를 푸는 두 방법이 있습니다. (실습 파일: [sections/19_agent_memory/06_긴-대화-메시지-관리.py](#))

- (a) 트리밍(trimming): 오래된 메시지를 잘라 버린다 — "버리기"
- (b) 요약(summarization): 오래된 대화를 한 줄로 압축한다 — "압축하기"

3. 해법 (a) — 트리밍

최근 N개 메시지만 남기고 나머지는 버립니다. LangChain의 `trim_messages` 를 씁니다.

```
from langchain_core.messages import trim_messages

def trim_recent(messages, keep=4):
    """최근 메시지만 남긴다(시스템 메시지는 항상 유지)."""
    return trim_messages(
        messages,
        max_tokens=keep,          # 여기선 '개수'로 해석(token_counter=len)
        strategy="last",         # 최근(last) 우선 보존
        token_counter=len,       # 메시지 '개수' 기준
```



```

include_system=True,      # 시스템 메시지는 항상 유지
start_on="human",        # 사람 메시지에서 시작하도록 경계 정리
)

```

핵심 옵션:

- `strategy="last"`: 최근 것을 우선 보존
- `include_system=True`: 시스템 메시지(역할 지침)는 항상 유지 — 이게 사라지면 봇이 정체성을 잃음
- `token_counter=len`: 토큰 대신 메시지 개수로 자름(간단)

예상 출력

[원본] 메시지 13개 → [트리밍] 5개 (시스템 1 + 최근 4)

13개가 5개로 줄었습니다. 다음 호출부터 입력이 줄어 비용·지연이 감소하죠.

4. 해법 (b) — 요약 압축

트리밍은 오래된 정보를 완전히 버립니다. 하지만 그 안에 중요한 게 있었다면? 그래서 요약으로 압축하는 방법이 있습니다.

```

def summarize_old(messages, keep_recent=4):
    """최근 keep_recent개는 그대로, 그 이전 대화를 LLM으로 한 줄 요약해 치환."""
    sys_msgs = [m for m in messages if isinstance(m, SystemMessage)]
    convo = [m for m in messages if not isinstance(m, SystemMessage)]
    if len(convo) <= keep_recent:
        return messages

    old, recent = convo[:-keep_recent], convo[-keep_recent:]
    # 오래된 대화를 텍스트로 합쳐 LLM에 요약 요청
    transcript = "\n".join(f"{'사용자' if isinstance(m, HumanMessage) else '상담원'}: {m.content}"
                           for m in old)
    summary = get_chat(temperature=0).invoke(
        "다음 상담 대화를 한국어 한 문장으로 요약하라.\n\n" + transcript).content
    # 오래된 N개를 '요약 1개'로 치환 → 길이는 줄고 맥락은 유지
    압축 = SystemMessage(content=f"[이전 대화 요약] {summary}")
    return sys_msgs + [압축] + recent

```

오래된 대화를 LLM이 한 문장으로 요약해, 그 요약 한 줄로 대체합니다. 최근 대화는 그대로 두고요.

예상 출력

[압축] 메시지 13개 → 6개

- SystemMessage: 너는 승승장구물 CS 상담원이다.
- SystemMessage: [이전 대화 요약] 고객이 배송·환불·멤버십·쿠폰에 대해 문의함.
- HumanMessage: (최근 4개)
- ...

오래된 6턴이 "[이전 대화 요약] 배송·환불·멤버십 문의" 한 줄로 압축됐습니다. 길이는 줄었지만 맥락은 유지되죠.

5. 트리밍 vs 요약

방법	하는 일	장점	단점
트리밍	오래된 것 버리기	간단·빠름·공짜	정보 손실
요약	오래된 것 압축	맥락 보존	LLM 호출 비용

- 트리밍: 최근 대화만 중요할 때(예: 단순 Q&A). 간단하고 공짜.
- 요약: 오래된 맥락도 중요할 때(예: 긴 상담). 맥락 보존하지만 요약에 LLM 호출 비용.

상황에 맞게 고릅니다. 보통 "최근 N개는 그대로 + 그 이전은 요약"을 섞어 쓰면 균형이 좋죠.

6. InMemorySaver와의 관계

이 강의 InMemorySaver는 대화를 자동 누적합니다. 편하지만, 길어지면 위 문제가 생기죠. 실무에서는:

InMemorySaver로 자동 누적 (편함)
+ 일정 길이 넘으면 트리밍/요약 적용 (비용 통제)

자동 누적의 편리함을 쓰되, 길어지면 트리밍·요약으로 입력을 줄입니다.— "기억하되, 비용을 통제한다."

2. Long term memory

어제 말한 걸 기억 못 하는 봇

앞 장의 InMemorySaver 는 **인메모리**라, 서버를 재시작하면 모든 기억이 사라집니다. "저 선크림 알레르기 있어요" 같은 정보는 **다음에 접속해도** 반영돼야 하죠. 사용자에게 귀속되는 정보는 **영속 저장소(파일/DB)**에 남겨야 합니다.

1. 단기 기억의 한계

19장에서 InMemorySaver로 대화를 기억하게 만들었습니다. 하지만 한계가 있죠.

⚠ 프로세스가 재시작되면 모든 기억이 사라진다!

서버를 켜다 켜거나, 사용자가 며칠 뒤 다시 접속하면, 19장의 기억은 **다 날아갑니다**. 메모리에만 있었으니까요. 그래서 "오늘 대화"는 기억하지만 "어제 대화"는 못 합니다.

2. 영속이 필요한 정보들

실제 서비스에서는 세션을 넘어 기억해야 할 정보가 많습니다.

"저 민감성 피부라 무향 제품만 추천해 주세요."

→ 다음에 접속해도 이 사실이 반영돼야 함

"제 등급이 VIP인데요."

→ 매번 다시 물어보면 안 됨

"저는 선크림 알레르기가 있어요."

→ 화장품 추천 시 항상 고려돼야 함 (안전 문제!)

이런 정보는 **한 대화 세션이 아니라 사용자에게 영구히 귀속**됩니다. 며칠 뒤 다시 와도, 서버를 재시작해도 기억해야 하죠. 단기 기억으로는 부족합니다.

비유: 단기 기억은 상담 중 메모지(상담 끝나면 버림), 장기 기억은 고객 카드(파일)(영구 보관)입니다.
"이 고객은 알레르기 있음"은 메모지가 아니라 고객 카드에 적어야 다음에도 보죠.

3. 백엔드에겐 익숙한 일

이 구분은 백엔드 개발자에게 매우 익숙합니다.

세션 데이터(휘발성) → 메모리/세션 스토어 → 단기 기억 (19강)
사용자 레코드(영속) → 데이터베이스 → 장기 기억 (20강)

웹 서비스에서 "장바구니"는 세션에, "회원 정보"는 DB에 저장하죠. LLM 에이전트도 똑같습니다 — "지금 대화 맥락"은 메모리에(단기), "사용자 프로필"은 파일/DB에(장기). 데이터의 수명에 따라 저장 위치가 다른 겁니다.

핵심 정리

- 1 장의 InMemorySaver 는 **인메모리** → 재시작 시 소멸.
- 알레르기·선호·등급 등은 **세션을 넘어** 기억해야 함.
- 사용자에게 귀속되는 정보는 **영속 저장소(파일/DB)** 에.
- 백엔드의 **세션(휘발) vs DB(영속)** 분리와 같다.
- 핵심: 에이전트를 **새로 만들어도** 저장 정보가 유지.

단기 기억 vs 장기 기억

단기 기억은 한 대화 세션의 "맥락"(메모리, 휘발), 장기 기억은 사용자에게 귀속된 "프로필/지식"(파일·DB, 영속)입니다. 둘은 보완 관계라 보통 함께 씁니다.

1. 두 기억 비교

구분	단기 기억 (Short-Term)	장기 기억 (Long-Term)
범위	한 대화 세션	사용자 단위, 세션 초월
저장 위치	메모리(InMemorySaver)	파일/DB (영속)
수명	프로세스/세션 종료 시 소멸	명시적으로 지울 때까지 유지
예시	"방금 말한 이름"	"이 고객은 무향 선호"
백엔드 비유	HTTP 세션	사용자 테이블(DB row)

2. 단기 기억 = 맥락

단기 기억(19강)은 지금 이 대화의 흐름을 잡습니다.

사용자: "이어버드 재고 있어?"

봇: "225개 있습니다."

사용자: "그거 가격은?"

← "그거" = 방금 말한 이어버드

봇: "79,000원입니다."

← 맥락(이어버드)을 단기 기억으로 유지

"그거"가 무엇인지 알려면 방금 대화를 기억해야 하죠. 이게 단기 기억의 역할 — 대화의 맥락 유지입니다. 대화가 끝나면 이 맥락은 더 이상 필요 없으니 사라져도 됩니다.

3. 장기 기억 = 프로필/지식

장기 기억(20강)은 사용자에게 영구히 귀속된 정보입니다.

사용자: "저 선크림 알레르기 있어요"

(며칠 뒤, 새 대화)

사용자: "화장품 추천해줘"

봇: "알레르기를 고려해 무향 제품을 추천합니다" ← 며칠 전 정보를 기억!

"선크림 알레르기"는 대화 맥락이 아니라 그 사용자의 영구 속성입니다. 며칠 뒤, 새 세션에서도 기억해야 하죠. 이게 장기 기억 — 프로필/지식의 영구 보존입니다.

4. 둘은 함께 쓴다

단기와 장기는 대립이 아니라 보완입니다. 보통 함께 씁니다.

단기 기억: "지금 이 대화에서 무슨 얘기 중인가" (맥락)

장기 기억: "이 사용자가 누구이고 뭘 선호하나" (프로필)

→ 둘 다 있어야 자연스러운 상담

좋은 상담봇은:

- 단기로 "방금 말한 주문번호"를 기억하고,
- 장기로 "이 고객은 VIP에 무향 선호"를 기억합니다.

그래서 이 강의 에이전트도 둘 다 씁니다 — **InMemorySaver** (단기) + 프로필 도구(장기).

비유: 단기 기억은 **대화 중 머릿속 단기 기억**(방금 들은 전화번호), 장기 기억은 **수첩에 적어 둔 정보**(친구 생일)입니다. 사람도 둘을 함께 쓰죠 — 지금 대화는 머리로, 영구 정보는 수첩으로.

5. 수명에 따른 저장 위치

핵심 원칙: 데이터의 수명에 따라 저장 위치를 정한다.

수명이 짧음 (대화 끝나면 불필요) → 메모리 (단기, 빠르고 휘발)
수명이 김 (영구 보존 필요) → 파일/DB (장기, 느리지만 영속)

"방금 말한 것"은 메모리에(빠름), "사용자 프로필"은 파일/DB에(영속). 백엔드에서 캐시(휘발)와 DB(영속)를 구분하는 것과 같습니다. 데이터의 성격을 보고 적절한 곳에 저장하는 거죠.

6. 장기 기억의 구현 방식

장기 기억을 구현하는 방법은 여러 가지입니다. 이 강은 **도구(Tool)**로 구현합니다.

방법 1: 도구로 (이 강) — 에이전트가 필요할 때 저장소를 읽고/쓴다
방법 2: 자동 주입 — 매 대화에 프로필을 자동으로 끼워 넣는다

도구 방식을 쓰는 이유: 에이전트가 "지금 프로필을 저장/조회해야겠다"를 스스로 판단하게 하면 가장 명확하고 제어 가능합니다(17강 "필요할 때만"의 연장). 다음 절에서 이 도구 방식을 자세히 봅니다.

핵심 정리

- **단기 기억** = 세션 맥락(메모리, 휘발, "방금 말한 것").
- **장기 기억** = 사용자 프로필/지식(파일·DB, 영속, "선호·알레르기").
- **둘은 보완** — 좋은 붓은 함께 쓴다.
- **데이터 수명**에 따라 저장 위치 결정(캐시 vs DB).
- 장기 기억을 **도구**로 구현(에이전트가 필요할 때 읽고/쓰기).

도구로서의 장기 기억

장기 기억을 읽기 도구(`get_profile`)와 쓰기 도구(`update_profile`)로 구현합니다. 에이전트가 "지금 프로필을 저장/조회해야겠다"를 스스로 판단해 도구를 호출하죠.

저장 계층(JSON→DB)을 바꿔도 도구 인터페이스만 유지하면 됩니다.

1. 두 도구

장기 기억은 두 도구로 구현합니다.

도구	역할	입력 → 출력
<code>get_profile</code>	프로필 조회(읽기)	고객ID → 이름/성호/비고
<code>update_profile</code>	프로필 갱신(쓰기)	고객ID, 필드, 값 → 저장 결과

읽기와 쓰기, 두 가지죠. 18강에서 배운 "읽기 도구는 안전, 쓰기 도구는 신중"이 여기서도 적용됩니다.

`update_profile`은 파일에 쓰니까요.

2. 동작 흐름

[저장]

- 사용자: "나 선크림 알레르기 있어. 기억해줘."
- 에이전트: "프로필에 저장해야겠다" → `update_profile(C0001, "비고", "선크림 알레르기")`
- `user_profiles.json` (영속 저장소)에 "선크림 알레르기" 기록됨

[조회 — 며칠 뒤, 새 세션]

- 사용자: "나한테 맞는 거 추천해줘"
- 에이전트: `get_profile(C0001)` → "선크림 알레르기"를 읽어 추천에 반영

쓰기(저장)와 읽기(조회)가 영속 저장소를 통해 세션을 넘어 연결됩니다. 며칠 전 쓴 정보를 오늘 읽는 거죠.

3. 에이전트가 스스로 판단

핵심은 에이전트가 언제 저장/조회할지 스스로 판단한다는 것입니다. 시스템 프롬프트로 지침을 줍니다.

```
system_prompt=(
    "너는 승승장구물 상담원이다. 한국어 존댓말로 답하라. "
    "고객이 자신의 선호/특이사항(알레르기 등)을 말하면 update_profile로 저장하라. "
    "추천이나 응대 전에는 get_profile로 고객 정보를 먼저 확인하라."
)
```

- "선호/특이사항을 말하면" → `update_profile` 로 저장
- "추천/응대 전에는" → `get_profile` 로 조회

이러면 에이전트가 대화를 보고 "아, 이건 저장할 정보네", "추천 전에 프로필 봐야겠네"를 스스로 판단합니다. — 필요할 때만 기억을 읽고 씁니다.

4. 왜 도구 방식인가

"매 대화에 프로필을 자동으로 끼워 넣으면 안 되나?"라고 생각할 수 있습니다. 도구 방식의 장점:

- 제어 가능: 에이전트가 "언제" 읽고 쓸지 명확 (디버깅 쉬움)
- 효율: 필요할 때만 저장소 접근 (매번 전체 프로필 주입 안 함)
- 선택적: 저장할 가치 있는 정보만 저장 (모든 발화를 저장 안 함)

자동 주입은 모든 대화에 프로필을 끼워 토큰을 낭비하고, 언제 저장됐는지 불투명합니다. 도구 방식은 에이전트의 판단에 따라 필요한 순간에만 작동하죠.

5. 저장 계층 분리 — 의존성 역전

가장 중요한 설계 원칙입니다. 도구 인터페이스(시그니처)는 고정하고, 내부 저장 방식만 바꿀 수 있게 합니다.

```
@tool
def get_profile(customer_id: str) -> str: # ← 이 시그니처는 고정
    """프로필 조회."""
    data = _load() # ← 내부 구현 (JSON? SQLite? 바뀌도 됨)
    ...
```

`get_profile` / `update_profile` 의 겉모습(시그니처) 은 그대로 두고, 내부의 `_load` / `_save` 만 JSON→SQLite→PostgreSQL로 교체할 수 있습니다(06번). 그러면 에이전트 코드는 전혀 안 바뀌도 저장소가 업그레이드되죠.

이걸 **의존성 역전**이라고 합니다. 에이전트는 "프로필을 읽고 쓴다"는 추상적 인터페이스에만 의존하고, "JSON 파일이나 DB냐"는 세부 구현을 모릅니다. 그래서 구현을 자유롭게 바꿀 수 있죠. 백엔드의 Repository 패턴과 같은 발상입니다.

6. 원본 보호

쓰기 도구는 위험하므로(18강), 실습에서는 원본을 보호합니다.

```
SRC = DATA / "user_profiles.json"      # 원본 (읽기 전용)
STORE = DATA / "user_profiles_rw.json" # 복사본 (실제 읽고/쓰기)
```

원본(**SRC**)은 건드리지 않고, 복사본(**STORE**)에만 읽고 씁니다. 실습 중 잘못 써도 원본이 안전하죠(18강 "복사본에만 쓰기"). 운영에서도 "마이그레이션으로 초기 데이터를 만들고, 운영 DB에서 변경"하는 식으로 분리합니다.

핵심 정리

- 장기 기억 = **get_profile(읽기)** + **update_profile(쓰기)** 두 도구.
- 에이전트가 **언제 저장/조회할지** 스스로 판단(시스템 프롬프트 지침).
- 도구 방식: **제어 가능·효율·선택적**(자동 주입보다 명확).
- **의존성 역전**: 도구 시그니처 고정, 내부 저장 방식만 교체(JSON→DB).
- 쓰기는 위험하니 **원본 보호**(복사본에 쓰기).

따라하기 — 프로필 영속 저장 도구

목표

사용자 프로필을 JSON 파일에 읽고 쓰는 장기 기억 도구(**get_profile**, **update_profile**)를 만듭니다. 실제 파일은 `code/ch20_ltm.py` 입니다.

0. 실습 준비

추가 설치 없음(JSON은 파이썬 표준). 데이터는 `data/user_profiles.json` (C0001 임도현, C0002 이서윤)을 씁니다.

```
conda activate agentic
python code/common.py
```

원본을 보호하기 위해 복사본(`user_profiles_rw.json`)에 읽고 씁니다. 없으면 코드가 자동 생성합니다.

1. 영속 저장소 도우미 (load/save)

JSON 파일을 읽고 쓰는 헬퍼입니다. 03번의 "원본 보호" 패턴입니다.

```
import json, shutil
from common import DATA

SRC = DATA / "user_profiles.json"      # 원본(읽기 전용)
STORE = DATA / "user_profiles_rw.json" # 실제 읽고/쓰는 복사본(영속 저장소)

def _load() -> dict:
    """저장소(JSON)를 읽어 dict로 반환. 없으면 원본을 복사해 초기화."""
    if not STORE.exists():
        shutil.copy(SRC, STORE)          # 최초 실행 시 원본 복사
    with open(STORE, encoding="utf-8") as f: # 한글이라 인코딩 명시
        return json.load(f)

def _save(data: dict) -> None:
    """dict를 저장소(JSON)에 영속 저장."""
    with open(STORE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)
```

두 가지 포인트:

- `encoding="utf-8"`: 한글 프로필이라 인코딩을 명시(안 하면 깨질 수 있음)
- `ensure_ascii=False`: JSON 저장 시 한글을 `\uXXXX` 가 아니라 그대로 저장(읽기 편함)

이 `_load` / `_save` 가 03번에서 말한 "내부 구현"입니다. 나중에 SQLite로 바꿀 부분이죠(06번).

2. 읽기 도구 — get_profile

```
from langchain_core.tools import tool

@tool
def get_profile(customer_id: str) -> str:
    """고객ID(예: C0001)로 저장된 프로필(이름/선호카테고리/멤버십/관심사/비고)을 조회한다."""
```

```

data = _load()
p = data.get(customer_id)
if not p:
    return f"{customer_id} 프로필이 없습니다." # 없어도 안전한 메시지
return json.dumps(p, ensure_ascii=False)

```

저장소를 읽어 해당 고객의 프로필을 돌려줍니다. 읽기 도구라 **안전하죠**(데이터 안 바꿈). 없는 고객이어도 예외 없이 안내합니다(도구 원칙).

3. 쓰기 도구 — update_profile

```

@tool
def update_profile(customer_id: str, field: str, value: str) -> str:
    """고객 프로필의 한 필드를 갱신해 영속 저장한다.
    field 예: '비고', '관심사', 'membership'."""
    # [장기 기억의 핵심] 이 도구가 파일에 '쓰기' 때문에 세션·재시작을 넘어 정보가 남는다
    data = _load()
    if customer_id not in data:
        data[customer_id] = {"name": "", "선호카테고리": [], "membership": "",
                             "관심사": "", "비고": ""}
    data[customer_id][field] = value
    _save(data) # ← 여기서 파일에 영속 저장!
    return f"{customer_id}의 '{field}'을(를) '{value}'(으)로 저장했습니다."

```

가장 중요한 줄은 `_save(data)` 입니다. 이게 파일에 쓰기 때문에, 정보가 세션·재시작을 넘어 남습니다. 이게 장기 기억의 본질이죠 — 메모리가 아니라 **파일(영속)**에 쓰는 것.

흐름: 저장소 읽기 → 해당 필드 수정 → 저장소에 쓰기. 없는 고객이면 빈 프로필을 먼저 만들고 수정합니다.

4. 도구 단독 테스트

에이전트에 붙이기 전 직접 호출해 봅니다.

```

print(get_profile.invoke({"customer_id": "C0001"})) # 기존 프로필 조회
print(update_profile.invoke({"customer_id": "C0001", "field": "비고", "value": "무향 선호"}))
print(get_profile.invoke({"customer_id": "C0001"})) # 변경 확인

```

예상 출력

```
{"name": "임도현", "선호카테고리": ["가전"], ...}  
C0001의 '비고'을(를) '무항 선호'(으)로 저장했습니다.  
{"name": "임도현", ..., "비고": "무항 선호"} ← 저장됨!
```

`update_profile` 후 `get_profile`로 확인하니 "무항 선호"가 저장됐습니다. 그리고 이건 **파일에 쓰였으니**, 프로그램을 껐다 켜도 남아 있죠(05번에서 확인).

5. JSON 파일 직접 확인

실제 파일을 열어 보면 변경이 디스크에 기록된 걸 볼 수 있습니다.

```
import json  
print(json.dumps(_load()["C0001"], ensure_ascii=False, indent=2))
```

예상 출력

```
{  
  "name": "임도현",  
  "선호카테고리": ["가전"],  
  "membership": "VIP",  
  "비고": "무항 선호"  
}
```

`user_profiles_rw.json` 파일에 실제로 "무항 선호"가 적혀 있습니다. 메모리가 아니라 **디스크 파일**에 있으니, 영속적이죠. 이게 단기 기억(메모리)과의 결정적 차이입니다.

핵심 정리

- `_load/_save` 로 JSON 파일을 읽고 쓴다(encoding·ensure_ascii 주의).
- `get_profile`(읽기, 안전), `update_profile`(쓰기, 영속).
- 핵심은 `_save(data)` — **파일에 쓰기** 때문에 세션·재시작을 넘어 남음.
- **원본 보호**(SRC) + 복사본에 쓰기(STORE).
- 변경이 **디스크 파일**에 기록 → 영속(메모리와 다름).

따라하기 — 새 세션에서도 기억

목표

장기 기억 도구를 에이전트에 붙이고, 에이전트를 새로 만든 뒤(=재시작)에도 저장한 정보가 살아 있는지 확인합니다. 이게 장기 기억의 본질입니다.

1. 장기 + 단기 기억 함께

에이전트에 장기 기억 도구(get/update_profile)와 단기 기억(InMemorySaver)을 둘 다 붙입니다(02번).

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from common import get_chat

def build_agent():
    llm = get_chat(temperature=0)
    agent = create_agent(
        llm,
        tools=[get_profile, update_profile], # 장기 기억 (파일)
        system_prompt=(
            "너는 승승장구물 상담원이다. 한국어 존댓말로 답하라. "
            "고객이 자신의 선호/특이사항(알레르기 등)을 말하면 update_profile로 저장하라. "
            "추천이나 응대 전에는 get_profile로 고객 정보를 먼저 확인하라."
        ),
        checkpointer=InMemorySaver(), # 단기 기억 (세션 맥락)
    )
    return agent

def say(agent, text, thread_id):
    config = {"configurable": {"thread_id": thread_id}}
    out = agent.invoke({"messages": [{"role": "user", "content": text}], config)
    return out["messages"][-1].content
```

장기(도구)로 세션을 넘긴 사실을 보존하고, 단기(InMemorySaver)로 한 세션의 맥락을 잊습니다. 둘이 협력 하죠.

2. 1단계 — 알레르기 정보 저장

첫 상담에서 알레르기를 알려줍니다.

```
agent = build_agent()
print(say(agent, "저 C0001 임도현인데요, 선크림 알레르기 있어요. 기억해 주세요.", "sess-1"))
```

예상 출력

네, 임도현님. 선크림 알레르기가 있으신 점 프로필에 저장해 두었습니다.

에이전트가 "선호/특이사항을 말하면 저장하라"는 지침에 따라, `update_profile` 을 호출해 파일에 저장했습니다. 이제 이 정보는 디스크에 있죠.

3. 2단계 — 핵심 실험: 새 에이전트로 조회

이게 이 강의 핵심 실험입니다. 에이전트를 새로 만들고(=프로세스 재시작에 해당), 다른 `thread_id`로 물어봅니다.

```
# 새 에이전트 = 단기 기억 완전히 비워짐 (재시작 시뮬레이션)
agent2 = build_agent()
print(say(agent2, "C0001 고객에게 화장품 추천할 때 주의할 점 있어?", "sess-2"))
```

예상 출력

C0001 임도현님은 선크림 알레르기가 있으시니, 자외선 차단 제품 추천 시
성분을 반드시 확인하고 대체 제품을 안내해야 합니다. ← 새 세션인데 기억!

놀라운 점: `agent2` 는 완전히 새 에이전트입니다. 단기 기억(InMemorySaver)은 텅 비었고, `thread_id`도 다릅니다(`sess-2`). 그런데도 "선크림 알레르기"를 기억하고 답했죠!

4. 왜 기억했나 — 파일 덕분

비결은 파일(영속 저장소)입니다.

1. `agent` (1단계) → `update_profile` → `user_profiles_rw.json` 에 저장 (파일에 남음)
2. `agent2` (2단계, 새 에이전트) → `get_profile` → 파일에서 "알레르기" 읽음 → 답변

1 단계에서 파일에 쓴 정보가, 2 단계의 새 에이전트가 get_profile 로 읽었습니다. 에이전트는 새것이지만, **파일은 그대로** 있었으니까요.

만약 이게 단기 기억(InMemorySaver)만이었다면? 새 에이전트는 텅 비어서 "그런 정보 없습니다"라고 했을 겁니다. 하지만 장기 기억은 **파일에 영속**되니, 에이전트가 바뀌어도 살아남죠. 이게 단기와 장기의 결정적 차이입니다.

5. 파일로 검증

저장소 파일을 직접 열어 확인합니다.

```
import json
print(json.dumps(_load()["C0001"], ensure_ascii=False, indent=2))
```

예상 출력

```
{
  "name": "임도현",
  "선호카테고리": ["가전"],
  "membership": "VIP",
  "비고": "주말 배송 선호 / 선크림 알레르기 있음"
}
```

파일에 "선크림 알레르기"가 기록돼 있습니다. 이건 메모리가 아니라 디스크라, 프로그램을 완전히 종료해도 남아 있죠. 다음에 프로그램을 다시 켜도, 이 파일을 읽으면 알레르기 정보가 그대로입니다.

6. 이것이 장기 기억의 본질

이 실험이 보여 주는 것:

에이전트(메모리)는 휘발해도, 파일(영속)에 쓴 정보는 살아남는다.

단기 기억(InMemorySaver): 에이전트와 함께 소멸 → "방금 대화"만
장기 기억(파일/DB): 에이전트와 무관하게 영속 → "사용자 프로필"

사용자가 며칠 뒤 다시 와도, 서버를 재시작해도, 파일에 쓴 "알레르기·선호·등급"은 그대로 기억됩니다. 이게 진짜 "사용자를 기억하는" 서비스의 모습이죠.

핵심 정리

- 장기 도구(파일) + 단기(InMemorySaver)를 **함께** 붙인다.
- 핵심 실험: **새 에이전트**(재시작)로도 저장 정보가 살아 있는지 확인.
- 새 에이전트인데 기억한 이유는 **파일(영속)**에 남았기 때문.
- 단기는 에이전트와 함께 소멸, 장기는 **파일에 영속**.
- 디스크 파일은 프로그램 종료·재시작에도 유지.

SQLite 로 장기 기억 옮기기

JSON 파일은 매번 **전체**를 읽고 씁니다.

데이터가 커지거나 동시 접근이 생기면 위험하죠.

SQLite(설치 없이 파일 하나로 쓰는 내장 DB)로 옮기면, 레코드 단위 조회·갱신이 가능해 안전합니다.

핵심은 **도구 인터페이스**는 그대로 두고 **내부만 교체**하는 것입니다.

1. JSON의 한계

04번의 JSON 방식은 간단하지만 문제가 있습니다.

JSON: 한 명의 비교를 바꾸려 해도
→ 파일 전체를 읽고(_load) → 수정 → 파일 전체를 씀(_save)

데이터가 작으면 괜찮지만:

- **데이터가 크면**: 고객 10만 명이면 매번 10만 명 전체를 읽고 씀(느림)
- **동시 접근**: 두 요청이 동시에 쓰면 한쪽 변경이 덮어써짐(데이터 손실)
- **부분 조회 불가**: "VIP 고객만" 같은 조건 검색이 어려움

실무 규모에서는 **레코드(행) 단위**로 다루는 **DB**가 필요합니다.

2. SQLite — 파일 하나짜리 DB

SQLite는 설치 없이 파일 하나로 쓰는 **내장 데이터베이스**입니다. 파이썬 표준 라이브러리(`sqlite3`)에 들어 있어 바로 쓸 수 있죠.

JSON: profiles.json (전체를 한 덩어리로)
SQLite: profiles.db (행 단위로 조회 · 갱신 가능, 동시성 안전)

서버를 띄울 필요도, 설치할 것도 없습니다. 파일 하나가 곧 DB죠. 소규모~중규모에 딱 좋고, 나중에 PostgreSQL 같은 큰 DB로 가는 디딤돌이 됩니다.

3. 테이블 만들기 + JSON 이관

(실습 파일: [sections/20_long_term_memory/06_SQLite로-장기기억-옮기기.py](#)) 테이블을 만들고, 기존 JSON 데이터를 한 번 옮깁니다.

```
import sqlite3, json
from common import DATA

DB = DATA / "profiles.db"

def init_db():
    """profiles 테이블을 만들고, 비어 있으면 JSON 원본을 한 번 이관한다."""
    with sqlite3.connect(DB) as conn:
        # customer_id를 기본키(PK)로 → 고객당 한 행만 존재
        conn.execute(
            "CREATE TABLE IF NOT EXISTS profiles ("
            " customer_id TEXT PRIMARY KEY, name TEXT, membership TEXT,"
            " 선호카테고리 TEXT, 관심사 TEXT, 비고 TEXT)"
        )
        # 비어 있으면 JSON → SQLite 이관 (최초 1회)
        if conn.execute("SELECT COUNT(*) FROM profiles").fetchone()[0] == 0:
            data = json.load(open(DATA / "user_profiles.json", encoding="utf-8"))
            for cid, p in data.items():
                conn.execute("INSERT INTO profiles VALUES (?, ?, ?, ?, ?, ?)",
                    (cid, p.get("name", ""), p.get("membership", ""),
                     json.dumps(p.get("선호카테고리", []), ensure_ascii=False),
                     p.get("관심사", ""), p.get("비고", "")))
```

- **PRIMARY KEY:** `customer_id`를 기본키로 뒤 고객당 한 행만(중복 방지)
- 선호카테고리는 **JSON 문자열**로: 리스트라 컬럼 하나에 담기 위해 `json.dumps`로 직렬화

4. 도구를 SQLite로 재구현

핵심: 도구 시그니처는 04번과 똑같이 두고, 내부만 SQLite로 바꿉니다(03번 의존성 역전).

```
@tool
def get_profile(customer_id: str) -> str: # ← 시그니처는 04번과 동일!
    """고객ID로 SQLite에 저장된 프로필을 조회한다."""
    with sqlite3.connect(DB) as conn:
        conn.row_factory = sqlite3.Row # row["컬럼명"]으로 접근
        row = conn.execute("SELECT * FROM profiles WHERE customer_id=?",
                           (customer_id,)).fetchone()

    if not row:
        return f"{customer_id} 프로필이 없습니다."
    p = dict(row)
    p["선호카테고리"] = json.loads(p["선호카테고리"] or "[]") # 다시 리스트로
    return json.dumps(p, ensure_ascii=False)
```

`SELECT ... WHERE customer_id=?`로 그 고객 한 행만 조회합니다. JSON처럼 전체를 읽지 않죠. 데이터가 많아도 빠릅니다(특히 PK 인덱스 덕분).

도구의 겉모습(`get_profile(customer_id) -> str`)은 04번과 똑같습니다. 그래서 에이전트 코드는 전혀 안 바뀌도 됩니다. 저장소만 JSON→SQLite로 업그레이드한 거죠.

5. SQL 인젝션 방어

쓰기 도구에서 중요한 보안 포인트가 있습니다.

```
@tool
def update_profile(customer_id: str, field: str, value: str) -> str:
    # [경고] 컬럼명(field)을 외부 입력 그대로 SQL에 끼우면 인젝션 위험!
    allowed = {"name", "membership", "선호카테고리", "관심사", "비고"}
    if field not in allowed: # 허용 컬럼만 화이트리스트
        return f"'{field}'은(는) 갱신할 수 없는 필드입니다."
    with sqlite3.connect(DB) as conn:
        conn.execute(f"UPDATE profiles SET {field}=? WHERE customer_id=?",
                    (value, customer_id)) # value는 ? 파라미터로 (안전)
    return f"저장했습니다."
```

두 가지 방어:

- 컬럼명(**field**)은 화이트리스트: `field`를 SQL에 직접 끼우면 위험하니, 허용된 컬럼만 통과
- 값(**value**)은 `?` 파라미터: 값을 SQL에 직접 안 넣고 `?`로 바인딩(SQL 인젝션 방지)

`value`를 `f"... SET {field}='{value}'"`처럼 직접 끼우면, 악의적 입력으로 DB를 조작할 수 있습니다(보안 정신).

`?` 파라미터를 쓰면 SQLite가 안전하게 처리하죠. 쓰기 도구는 이렇게 신중해야 합니다.

6. 같은 실험, 같은 결과

SQLite로 바뀌도 05번의 실험은 똑같이 작동합니다.

1단계: `update_profile` → SQLite DB에 저장
2단계: 새 에이전트 → `get_profile` → DB에서 읽어 답변 (기억!)

저장소가 JSON이든 SQLite든, "새 에이전트도 기억한다"는 장기 기억의 본질은 같습니다. 단지 SQLite가 동시성·조화·확장에 더 강할 뿐이죠. 그리고 도구 시그니처를 고정한 덕분에, 이 교체에 에이전트 코드 변경이 전혀 없었습니다.

7. 다음 단계 — PostgreSQL 등

SQLite도 한계가 있습니다. 여러 서버가 동시에 접근하거나, 초대규모면 **PostgreSQL·MySQL** 같은 서버형 DB가 필요하죠.

소규모 · 단일 서버 → SQLite (파일 하나, 간단)
대규모 · 다중 서버 → PostgreSQL/MySQL (서버형, 강력)

하지만 원리와 도구 인터페이스는 그대로입니다. `get_profile` / `update_profile`의 내부를 PostgreSQL 쿼리로 바꾸기만 하면 되죠. 의존성 역전 덕분에, 저장소는 규모에 맞게 자유롭게 갈아탑니다.

핵심 정리

- JSON은 **매번 전체**를 읽고 써 대규모·동시 접근에 약함.
- **SQLite**(파일 하나 내장 DB)로 **행 단위** 조화·갱신 → 안전·빠름.
- **도구 시그니처 고정, 내부만 교체**(의존성 역전) → 에이전트 코드 불변.
- 보안: 컬럼명은 **화이트리스트**, 값은 ? **파라미터**(SQL 인젝션 방지).
- 더 큰 규모는 PostgreSQL 등 — 원리·인터페이스는 동일.

3. Research agent

최신 시장 동향을 매번 사람이 검색?

승승장구물 MD 팀은 "경쟁 무선이어버드 시장 동향"을 주기적으로 파악해야 합니다.

사람이 검색 → 여러 페이지 읽기 → 요약 → 문서화 하는 과정은 반복적이고 시간이 많이 듭니다. 게다가 LLM은 학습 시점 이후의 최신 정보를 모릅니다(지식 컷오프).

이번 장에서는 이 일을 자동화하는 리서치 에이전트를 만듭니다.

1. 반복되는 수작업

MD(상품기획) 담당자의 월요일 아침을 떠올려 봅시다.

1. 구글에 "무선이어버드 시장 동향 2026" 검색
2. 블로그 · 뉴스 · 리포트 10개 클릭해서 읽기
3. 핵심만 추려서 메모
4. 메모를 정리해 보고서 한 장 작성
5. 다음 주에 또 반복...

이 일은 중요하지만 반복적입니다. 그리고 사람마다 검색어·정리 방식이 달라 품질이 들쭉날쭉하죠. "매주 같은 양식의 시장 리포트"가 필요한데, 매번 사람이 처음부터 하는 건 비효율입니다.

2. LLM 혼자서는 안 되는 이유

"그냥 LLM한테 물어보면 되지 않나요?" 라고 생각할 수 있습니다. 직접 해 봅시다.

```
# LLM에게 최신 정보를 그냥 물어보면?
from common import get_chat

llm = get_chat()
answer = llm.invoke("2026년 무선이어버드 시장의 최신 동향을 알려줘.").content
print(answer)
```

예상 출력(문제 상황)

무선이어버드 시장은 ANC(노이즈 캔슬링)와 ... 으로 성장하고 있습니다.
(단, 저는 특정 시점까지 학습되어 최신 수치나 신제품 정보는 정확하지 않을 수 있습니다.)

LLM은 학습 시점(지식 컷오프) 이후의 일을 모릅니다. 어제 나온 신제품, 이번 주 가격 변동, 최근 발표된 시장 점유율 — 이런 건 답하지 못하거나, 심하면 그럴듯하게 지어냅니다(환각). 최신성이 생명인 시장 조사에서는 치명적이죠.

3. 해결책 — 검색으로 사실에 근거(grounding)

답은 간단합니다. LLM에게 웹 검색 도구를 쥐여 주는 것입니다.

[LLM 혼자] 질문 → (기억나는 대로 답) → 최신성 · 정확성 불안
[LLM + 검색 도구] 질문 → 웹 검색 → 실제 결과를 읽고 → 사실 기반 답

이것이 5강(Function Calling)·9강(검색 에이전트)에서 배운 그라운드링(grounding)입니다. LLM의 "말 잘하는 능력"은 살리되, 사실은 웹에서 실시간으로 가져와 답을 근거에 묶습니다. 최신 스니펫을 읽고 요약하니, 지식 컷오프 문제가 사라지죠.

4. 우리가 만들 것 — 리서치 에이전트

이번 강의 목표물은 다음과 같습니다.

1. 주제 입력: "경쟁 무선이어버드 시장 동향"
2. 리서치 에이전트 — 웹 검색 도구를 스스로 사용 (검색 → 결과 수집 → 요약)
3. 마크다운 리포트(.md) 자동 생성·저장 — ## 개요 / ## 핵심 동향 / ## 시사점

사람이 하던 "검색→읽기→요약→문서화"를 함수 한 번 호출로 끝냅니다. 그리고 결과를 .md 파일로 떨어뜨려, 매주 같은 양식의 리포트를 배치 잡으로 자동 생성할 수 있게 만듭니다.

5. 백엔드 관점 — 데이터 파이프라인

개발자 눈으로 보면 리서치 에이전트는 전형적인 데이터 파이프라인입니다.

외부 API 호출(검색) → 데이터 정제(요약) → 산출물 생성(.md 저장)
↑ 네트워크 의존 ↑ LLM 처리 ↑ 파일 I/O

- 정기 배치: 매주 월요일 새벽에 시장 리포트 자동 생성
- 온디맨드 API: "/research?topic=..." 엔드포인트로 즉석 조사
- 격리·폴백: 검색이 막혀도(rate limit·네트워크 단절) 서비스가 죽지 않게

핵심 정리

- 시장 조사는 **반복적·시간 소모적**이고 사람마다 품질이 다름.
- LLM 혼자서는 **지식 컷오프**로 최신 정보를 모르고, 심하면 지어냄(환각).
- **웹 검색 도구**로 사실에 근거(grounding)시키면 최신성 문제 해결.
- 목표물: 주제 → 검색·요약 → **마크다운 리포트 자동 생성**.
- 백엔드로는 "외부 API → 정제 → 산출물"의 **데이터 파이프라인**(배치·API).

리서치 에이전트 패턴

리서치 에이전트는 **검색 → 수집 → 요약 → 리포트** 4단계로 동작합니다. 이번 강에서는 create_agent에 검색 도구를 붙여, **에이전트가 스스로 검색하고 요약**하게 한 뒤 결과를 .md 로 저장합니다.

1. 4단계 패턴

리서치 에이전트의 표준 흐름은 다음과 같습니다.

조사 주제가 들어오면 **검색 → 수집 → 요약 → 리포트** 순서로 흐릅니다.

단계	하는 일	구현
검색	키워드로 웹 질의	<code>DuckDuckGoSearchResults</code>
수집	결과 스니펫/링크 모음	도구 반환값
요약	잡음 제거·핵심 정리	LLM
리포트	사람이 읽을 문서화	마크다운 + 파일 저장

핵심은 **검색(사실 수집)**과 **요약(글쓰기)**을 분업한다는 점입니다. 검색은 도구가, 요약은 LLM이 담당하죠.

2. 누가 검색을 결정하나 — 에이전트의 자율성

여기서 중요한 설계 선택이 있습니다. "검색을 언제, 몇 번 할지"를 누가 정하느냐입니다.

[방법 A] 사람이 코드로 강제: 검색() → 요약() (07강 수동 루프 스타일)

[방법 B] 에이전트가 스스로 판단: create_agent에 도구만 주고 맡김 ← 이번 강

이번 3장은 **방법 B**입니다. create_agent에 검색 도구를 넘기면, 에이전트가 스스로 "이 주제는 검색이 필요한군 → 검색 → 결과를 보니 부족하군 → 한 번 더 검색 → 이제 요약하자"를 반복합니다(ReAct 루프). 우리는 주제만 던지면 됩니다.

```
from langchain_community.tools import DuckDuckGoSearchResults
from langchain.agents import create_agent

agent = create_agent(
    llm,
    tools=[DuckDuckGoSearchResults()], # 검색 도구를 주면
    system_prompt="...시장 조사 애널리스트...",
)
# 검색 → 요약은 에이전트가 알아서 (우리는 주제만 던진다)
result = agent.invoke({"messages": [{"role": "user", "content": "...리포트 작성해줘"}]})
```

3. 시스템 프롬프트로 역할·출력 형식 고정

에이전트가 "아무 글"이나 쓰면 곤란합니다. 항상 같은 양식의 리포트가 나오도록 시스템 프롬프트로 역할과 출력 형식을 못박습니다(2강·3강).

```
system_prompt=(
    "너는 승승장구몰의 시장 조사 애널리스트다. "
    "주어진 주제를 웹에서 검색해 핵심 사실을 수집하고, "
    "한국어 마크다운 리포트로 정리하라. "
    "리포트는 '## 개요 / ## 핵심 동향 / ## 시사점' 섹션을 포함한다."
)
```

이렇게 하면 어떤 주제를 넣어도 **개요·핵심 동향·시사점** 세 섹션을 갖춘 리포트가 나옵니다. 출력 형식을 고정하는 건 자동화의 기본이죠 — 뒷단에서 파싱·저장·비교하기 쉬워지니까요.

이번 3장에서 만들 코드의 구조를 미리 봅니다.

- `build_research_agent()` — 검색 도구 붙인 에이전트 생성
- `research(topic)` — 에이전트 호출(검색·요약 자동), 실패 시 폴백
- `save_report(topic, body)` — 결과를 `reports/*.md` 로 저장

함수를 세 개로 분리한 이유는 재사용·테스트 때문입니다. 배치 스케줄러는 `research → save_report` 만 호출하면 되고, API는 `research()` 결과를 바로 JSON으로 응답할 수 있죠. 모듈화는 운영의 기본입니다.

핵심 정리

- 리서치 에이전트 = 검색 → 수집 → 요약 → 리포트 4단계.
- 검색은 도구(사실 수집), 요약은 LLM(글쓰기)으로 분업.
- create_agent에 도구를 주면 에이전트가 스스로 검색·요약(ReAct 루프).
- 시스템 프롬프트로 역할·출력 형식(섹션 구조)을 고정 → 자동화에 유리.

네트워크 의존성

검색은 외부 네트워크에 의존합니다. 네트워크 단절·검색 차단(rate limit)으로 언제든지 실패할 수 있죠. 리서치 에이전트는 try/except로 감싸고, 실패해도 빈손으로 끝내지 않는 폴백(fallback)을 반드시 갖춰야 합니다.

1. 검색은 언제든지 실패할 수 있다

LLM 호출은 우리 코드 안에서 돌지만, 웹 검색은 바깥 세상에 의존합니다.

내 코드 → (HTTP 요청) → 검색 서비스(DuckDuckGo) → 웹 곳곳

이 경로의 어느 지점에서든 끊기면 검색이 실패합니다.

실패 원인은 다양합니다.

- 네트워크 단절 (와이파이 끊김, 사내 방화벽)
- rate limit (짧은 시간에 너무 자주 호출 → 일시 차단)
- 검색 서비스 장애 (상대 서버 다운)
- 타임아웃 (응답이 너무 느림)

이건 내가 잘못해서가 아니라, 외부 의존성의 본질입니다. "가끔 실패한다"를 전제로 설계해야 하죠.

2. 실패하면 서비스가 죽는다 — 나쁜 예

폴백 없이 짜면 이렇게 됩니다.


```
# 나쁜 예: 검색 실패가 그대로 예외로 터진다
def research(topic):
    agent = build_research_agent()
    result = agent.invoke({"messages": [{"role": "user", "content": f"{topic} 리포트"}]})
    return result["messages"][-1].text
# 검색이 rate limit에 걸리면 → 예외 발생 → 함수가 죽음 → 호출한 배치/API도 죽음
```

매주 월요일 새벽 배치가 돌다가 검색 차단 한 번에 통째로 실패하면? 그 주 리포트는 없습니다. API라면 사용자에게 500 에러가 가죠. "외부 한 곳의 실패가 내 서비스 전체를 멈춘다" — 이게 가장 흔한 운영 사고입니다.

3. 폴백 — 빈손으로 끝내지 않기

해결책은 **try/except + 폴백**입니다. 검색이 실패하면, 검색 없이 **LLM 지식만으로** 임시 리포트라도 돌려줍니다.

```
def research(topic: str) -> str:
    try:
        # 1순위: 검색 에이전트로 최신 정보 기반 리포트
        agent = build_research_agent()
        result = agent.invoke({"messages": [{"role": "user",
                                             "content": f"'{topic}' 시장 동향 리포트를 작성해줘."}]})
        return result["messages"][-1].text
    except Exception as e:
        # 폴백: 검색 실패 → LLM 지식만으로 임시 리포트(최신성은 떨어져도 일단 결과)
        print(f"[경고] 웹 검색 실패 → 오프라인 폴백 사용: {e}")
        llm = get_chat(temperature=0.3)
        prompt = (f"'{topic}'에 대해 아는 범위에서 한국어 마크다운 리포트를 작성하라. "
                  "'## 개요', '## 핵심 동향', '## 시사점' 세 섹션을 포함하라. "
                  "맨 위에 최신 정보가 아닐 수 있다는 주의문구 한 줄을 넣어라.")
        return llm.invoke(prompt).text
```

핵심 철학(5강·12강 폴백 정신):

```
최신(검색 기반 최신 리포트)이 안 되면
→ 차선(LLM 지식 기반 임시 리포트)이라도 돌려준다
→ 단, "최신이 아닐 수 있다"는 경고를 명시
```

빈손(에러)보다는 불완전해도 결과가 낫습니다. 단, 폴백 결과임을 사용자가 알 수 있게 경고 문구를 넣는 게 정직한 설계죠.

4. 운영에서 더 고려할 것

폴백 외에도 외부 API를 다룰 때의 실무 장치들입니다(29강 배포에서 심화).

- 타임아웃: 검색이 10초 넘게 안 오면 끊는다 (무한 대기 방지)
- 제한적 재시도: 일시적 실패면 1~2회 다시 (단, 무한 재시도는 금물)
- 로깅: 검색 실패율 · 폴백 발생 횟수를 기록 → 모니터링
- 격리: 검색 실패가 다른 기능(예: 주문 조회)까지 멈추지 않게

백엔드 관점: 리서치 에이전트는 정기 배치(매주 시장 리포트)나 온디맨드 엔드포인트로 노출하기 좋습니다. 단, 검색 실패가 전체 서비스 장애로 번지지 않도록 격리·폴백을 설계하세요. "외부 의존성은 반드시 실패한다"를 전제로 짜는 게 핵심입니다.

5. 폴백은 모든 외부 호출의 기본 패턴

이 패턴은 검색에만 쓰는 게 아닙니다. 외부에 의존하는 모든 호출에 적용됩니다.

- | | |
|------------|-----------------------------|
| 검색 API | → 실패 시 LLM 지식 폴백 (이번 강) |
| 결제 API | → 실패 시 "잠시 후 다시" 안내 + 재시도 큐 |
| DB 조회 | → 실패 시 캐시된 값 사용 |
| LLM API 자체 | → 실패 시 다른 모델 · 간단 응답으로 폴백 |

"외부에 무언가를 요청하는 코드"를 짤 때마다 "**이게 실패하면?**" 을 먼저 생각하는 습관 — 이게 견고한 백엔드 개발자의 기본기입니다.

핵심 정리

- 웹 검색은 **외부 네트워크 의존** → 단절·rate limit·타임아웃으로 언제든지 실패.
- 폴백 없이 짜면 **외부 한 곳의 실패가 서비스 전체를 멈춤**.
- **try/except + 폴백**: 검색 실패 시 LLM 지식만으로 임시 리포트(경고 명시).
- 빈손(에러)보다 **불완전해도 결과가 낫다**(폴백 정신).
- 타임아웃·재시도·로깅·격리도 함께 고려.

따라하기 — 리서치 에이전트와 폴백

목표

검색 도구를 붙인 리서치 에이전트를 만들고, 주제를 던지면 **검색→요약**으로 마크다운 리포트를 생성하게 합니다. 검색 실패 시에는 LLM 지식만으로 임시 리포트를 만드는 **폴백**을 붙입니다.

0. 실습 준비

```
conda activate agentic
pip install -U ddgs # DuckDuckGo 검색 백엔드 (이미 있으면 생략)
pip install -U langchain langchain-community langchain-google-genai
python code/common.py # 환경/키 점검
```

⚠ 이 실습은 **인터넷 연결이 필요합니다**. 오프라인이거나 검색이 차단된 환경에서는 아래 코드의 폴백 분기가 실행되어, 검색 없이 LLM 지식만으로 임시 리포트를 생성합니다. (즉, 네트워크가 없어도 코드는 끝까지 돌아갑니다.)

- `ddgs` : `DuckDuckGoSearchResults` 가 내부적으로 쓰는 검색 패키지. **API 키가 필요 없습니다**.
- `GOOGLE_API_KEY` : `.env` 에 있어야 LLM(Gemini)이 동작합니다.

1. 검색 도구를 붙인 에이전트

`code/ch22_research.py` 를 만듭니다. 먼저 임포트와 에이전트 빌더입니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch22_research.py
import sys, pathlib, datetime
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat

# DuckDuckGoSearchResults: 키가 필요 없는 무료 웹 검색 도구(ddgs 패키지 기반).
# [왜 검색 도구] LLM은 학습 시점 이후의 '최신 정보'를 모른다(지식 컷오프).
# 웹 검색으로 실시간 사실을 끌어와 답을 '사실에 근거(grounding)'시킨다.
from langchain_community.tools import DuckDuckGoSearchResults
from langchain.agents import create_agent # LLM+도구를 묶어 스스로 도구를 쓰는 에이전트 생성
```

```
def build_research_agent():
    """웹 검색 도구를 붙인 리서치 에이전트를 만들어 반환한다.

    create_agent 에 검색 도구를 넘기면, 에이전트가 스스로
    '검색 → 결과 수집 → 요약'을 반복해 리포트를 만든다.
    """
    llm = get_chat(temperature=0.3) # [왜 0.3] 사실 수집이지만 리포트 서술력도 필요해 약간 높임
    search = DuckDuckGoSearchResults() # 검색 도구 인스턴스(키 불필요)
    agent = create_agent(
        llm,
        tools=[search], # 이 도구를 언제 쓸지는 LLM이 스스로 판단
        system_prompt=( # 역할 · 출력형식을 고정하는 시스템 프롬프트
            "너는 승승장구몰의 시장 조사 애널리스트다. "
            "주어진 주제를 웹에서 검색해 핵심 사실을 수집하고, "
            "한국어 마크다운 리포트로 정리하라. "
            "리포트는 '## 개요 / ## 핵심 동향 / ## 시사점' 섹션을 포함한다."
        ),
    )
    return agent
```

짚어 둘 점

- `DuckDuckGoSearchResults()` — 키 없이 바로 쓰는 검색 도구. `ddgs` 패키지가 백엔드.
- `temperature=0.3` — 분류(0.0)와 달리 리포트는 서술이 필요해 약간 높였습니다. 너무 높이면 사실에서 벗어나니 0.3 정도가 적당.
- `tools=[search]` — 검색을 언제 할지는 LLM이 판단합니다(7강 ReAct). 우리는 도구만 쥐여 주죠.

2. 조사 실행 + 폴백 처리

검색은 외부 네트워크라 실패할 수 있습니다. `try/except` 로 감싸고, 실패 시 검색 없는 LLM으로 폴백합니다(03번).

```
def research(topic: str) -> str:
    """주제를 조사해 마크다운 리포트 본문(문자열)을 반환한다.

    [왜 try/except] 검색은 외부 네트워크에 의존한다. 단절 · rate limit 으로
    실패할 수 있으므로, 실패해도 서비스가 죽지 않도록 폴백 분기를 둔다.
    """
    try:
        agent = build_research_agent()
        # 에이전트 호출: messages 에 사용자 요청을 담아 보낸다(검색→요약은 내부에서 자동).
        result = agent.invoke({"messages": [{"role": "user",
            "content": f"'{topic}' 주제로 시장 동향 리포트를 작성해줘."}]})
        return result["messages"][-1].text # 마지막 메시지 = 최종 리포트 본문
```

```
except Exception as e:
    # 네트워크/검색 실패 → 검색 없이 LLM 지식만으로 임시 리포트(폴백)
    print(f"[경고] 웹 검색 실패 → 오프라인 폴백 사용: {e}")
    llm = get_chat(temperature=0.3)
    prompt = (f'{topic}'에 대해 아는 범위에서 한국어 마크다운 리포트를 작성하라. "
              "본문에 '## 개요', '## 핵심 동향', '## 시사점' 세 섹션을 정확히 이 머리말로 포함하라"
              "(머리말을 중복해서 붙이지 말 것). "
              "맨 위에 최신 정보가 아닐 수 있다는 주의문구 한 줄을 넣어라.")
    return llm.invoke(prompt).text
```

`result["messages"][-1].text` 가 핵심입니다. 에이전트는 대화 메시지 리스트를 돌려주는데, **마지막 메시지**가 최종 리포트입니다. `.text` 로 본문 문자열만 꺼냅니다.

`.text` vs `.content`: Gemini 응답은 내부적으로 여러 블록(content parts)으로 올 수 있어, 본문 텍스트만 안전하게 꺼내려면 `.text` 를 씁니다(19장에서 다룬 차이). 에이전트가 돌려준 메시지에서든 같은 원리입니다.

3. 실행해 보기

아직 `save_report` 는 05번에서 붙입니다. 일단 본문 생성까지 확인합니다.

```
if __name__ == "__main__":
    topic = "경쟁 무선이어버드 시장 동향"
    print(f"[조사 시작] {topic}")
    body = research(topic)      # 검색→요약(또는 폴백) → 리포트 본문
    print("=" * 60)
    print(body[:600], "...")   # 앞부분만 미리보기
    print("=" * 60)
```

예상 출력(검색 성공 시)

```
[조사 시작] 경쟁 무선이어버드 시장 동향
=====
## 개요
무선이어버드 시장은 ANC(능동소음제거)와 저지연 게이밍 모드를
중심으로 빠르게 성장하고 있다. ...

## 핵심 동향
- 프리미엄 ANC 보편화, 중저가 모델로 확산
- 멀티포인트 연결 · 공간음향 기능 경쟁 심화
- 배터리 · 케이스 무선충전이 기본 사양화
```

```
## 시사점
승승장구물 자체 PB 이어버드는 ANC와 멀티포인트를 ... 강화 필요.
...
```

예상 출력(검색 실패 → 폴백 시)

```
[조사 시작] 경쟁 무선이어버드 시장 동향
[경고] 웹 검색 실패 → 오프라인 폴백 사용: ...(rate limit 등)...

> ⚠ 아래 내용은 최신 정보가 아닐 수 있습니다(검색 미사용).

## 개요
...
```

관찰 포인트: 검색이 되면 최신 스니펫을 요약한 리포트가, 안 되면 폴백 리포트(경고 문구 포함)가 나옵니다. 어느 쪽이든 코드는 죽지 않고 결과를 돌려줍니다. 이게 폴백의 가치죠.

4. 에이전트가 내부에서 하는 일

`research()` 한 줄 뒤에서 에이전트는 이런 ReAct 루프를 돌립니다.

사용자: "경쟁 무선이어버드 시장 동향 리포트 써줘"

1. [생각] 최신 정보가 필요하다 → 검색하자
2. [행동] `DuckDuckGoSearchResults("무선이어버드 시장 동향 2026")`
3. [관찰] 스니펫 5개 수집 (ANC, 가격, 점유율 ...)
4. [생각] 정보가 충분하다 → 이제 섹션별로 요약하자
5. [행동] 개요/핵심동향/시사점 마크다운 작성
6. 최종 메시지 = 리포트 본문 → `research()` 가 이걸 반환

우리는 도구와 주제만 줬는데, 검색 횟수·검색어·요약 시점을 에이전트가 알아서 정했습니다. 이게 "자율 에이전트"의 힘이죠(반면 06번에서는 우리가 검색을 더 통제합니다).

핵심 정리

- `pip install ddgs` 후 `DuckDuckGoSearchResults()`로 키 없는 검색 도구 사용.
- `create_agent(llm, tools=[search], system_prompt=...)`로 리서치 에이전트 생성.
- `temperature=0.3` — 사실 기반이되 서술력도 필요해 약간 높임.
- `result["messages"][-1].text` — 마지막 메시지가 최종 리포트 본문.
- **try/except 폴백**: 검색 실패 시 LLM 지식만으로 임시 리포트(경고 명시).

따라하기 — 마크다운 리포트 저장

목표

04번에서 만든 리포트 본문을 헤더와 함께 `.md` 파일로 저장합니다. 조사(`research`)와 저장(`save_report`)을 분리해, 배치·API에서 재사용하기 쉬운 구조를 완성합니다.

1. 저장 함수 만들기

`code/ch22_research.py`에 `save_report`를 추가합니다.

```
def save_report(topic: str, body: str) -> pathlib.Path:
    """리포트 본문을 헤더와 함께 reports/ 폴더에 .md로 저장하고 경로를 반환한다.

    [왜 research/save 분리] 재사용 · 테스트를 쉽게 하려는 모듈화.
    배치는 research→save 만 호출하면 되고, API는 research() 결과를 바로 응답할 수 있다.
    """

    out_dir = pathlib.Path(__file__).resolve().parent.parent / "reports"
    out_dir.mkdir(exist_ok=True)      # reports/ 폴더가 없으면 생성
    today = datetime.date.today().isoformat()
    path = out_dir / f"research_{today}.md"
    header = f"# 시장 조사 리포트 — {topic}\n\n> 생성일: {today} (자동 생성)\n\n"
    path.write_text(header + body, encoding="utf-8")  # 한글 깨짐 방지 위해 utf-8 명시
    return path
```

짚어 둘 점

- `out_dir.mkdir(exist_ok=True)` — `reports/` 폴더가 없으면 만들고, 있으면 그냥 둡니다(`exist_ok=True`라 에러 안 남).
- `datetime.date.today().isoformat()` — `2026-06-07` 형태의 날짜 문자열. 파일명·헤더에 씁니다.
- `header` — 본문 위에 제목·생성일을 붙여 누가 봐도 무슨 리포트인지 알게 합니다.
- `encoding="utf-8"` — 한글이 깨지지 않게 인코딩을 명시합니다(매우 중요).

2. 전체 실행 코드

`research` (04번) → `save_report` (이번)를 이어 붙입니다.

```
if __name__ == "__main__":
```

```
topic = "경쟁 무선이어버드 시장 동향"
print(f"[조사 시작] {topic}")
body = research(topic)          # 검색→요약(또는 풀백) → 리포트 본문
path = save_report(topic, body)  # .md 파일로 저장
print("=" * 60)
print(body[:600], "...")        # 미리보기
print("=" * 60)
print("[저장 완료]", path)
```

예상 출력

[조사 시작] 경쟁 무선이어버드 시장 동향

```
## 개요
무선이어버드 시장은 ANC(능동소음제거)와 저지연 게이밍 모드를 ...
## 핵심 동향
- 프리미엄 ANC 보편화 ...
## 시사점
승승장구물 자체 PB 이어버드는 ...
```

[저장 완료] /Users/.../reports/research_2026-06-07.md

3. 저장된 파일 확인

생성된 `.md` 를 열어 봅니다.

```
cat reports/research_2026-06-07.md
```

파일 내용(예시)

```
# 시장 조사 리포트 — 경쟁 무선이어버드 시장 동향

> 생성일: 2026-06-07 (자동 생성)

## 개요
무선이어버드 시장은 ANC(능동소음제거)와 ...

## 핵심 동향
- 프리미엄 ANC 보편화, 중저가 모델로 확산
- 멀티포인트 연결 · 공간음향 기능 경쟁 심화
...
```


시사점

승승장구물 자체 PB 이어버드는 ANC와 멀티포인트를 ... 강화 필요.

헤더(제목·생성일)와 본문(세 섹션)이 합쳐진 **완성된 리포트**가 저장됐습니다. 이 `.md`는 그대로 사내 위키에 올리거나, HTML/PDF로 변환하거나, 메일로 보낼 수 있죠.

4. 왜 `research`와 `save`를 나눴나

함수를 둘로 나눈 건 **재사용·테스트** 때문입니다.

[배치 스케줄러] 매주 월요일 → `research(topic)` → `save_report(...)` → 파일 저장
[API 엔드포인트] GET `/research?topic=...` → `research(topic)` → JSON으로 바로 응답 (저장 안 함)
[테스트] `research()` 결과만 검증 (파일 I/O 없이 빠르게)

`research()`는 "본문 문자열 생성"만, `save_report()`는 "저장"만 합니다. **한 가지 일만 하는 함수**로 나누면 상황별로 조합해 쓰기 쉽죠(단일 책임 원칙). 만약 한 함수에 다 넣었다면, API에서 "저장은 필요 없는데 저장 코드가 끼어 있는" 어색한 구조가 됩니다.

5. 운영 모니터링 팁

리포트 생성을 배치로 돌릴 때, 다음을 로그로 남기면 좋습니다.

- 검색 호출 수 · 소요 시간 (성능 추적)
- 폴백 발생 여부 (검색이 자주 막히는지 모니터링)
- 생성된 파일 경로 · 크기 (산출물 확인)

```
import time
t0 = time.time()
body = research(topic)
elapsed = time.time() - t0
fallback = body.startswith("> ⚠") # 폴백 리포트인지 간단 판별
path = save_report(topic, body)
print(f"[로그] 소요={elapsed:.1f}s 폴백={fallback} 파일={path}")
```

이런 로그가 쌓이면 "검색이 자꾸 막히네 → 재시도/캐시 도입해야겠다" 같은 **운영 판단**을 데이터로 할 수 있습니다(배포).

핵심 정리

- `save_report()`는 본문에 헤더(제목·생성일)를 붙여 `reports/*.md`로 저장.
- `mkdir(exist_ok=True)`로 폴더 자동 생성, `encoding="utf-8"`로 한글 보존.
- `research(생성)`와 `save_report(저장)`를 분리 → 배치·API·테스트에서 재사용.
- 저장된 `.md`는 위키·HTML·메일 등으로 그대로 활용 가능.
- 소요 시간·폴백 발생을 로깅하면 운영 모니터링이 쉬움.

복잡한 주제를 하위 질의로 쪼개기

"무선이어버드 시장 동향"처럼 큰 주제를 검색어 하나로 던지면 결과가 얇습니다.

LLM이 주제를 3~4개 하위 질의(예: "ANC 트렌드", "가격대 동향", "주요 기업")로 분해하고, 각각을 따로 검색해 종합 리포트로 합칩니다.

이를 `plan-and-execute` 리서치라 합니다.

1. 검색어 하나의 한계

04번 에이전트는 검색을 알아서 했지만, 큰 주제는 한 번 검색으로는 얇게 다뤄집니다.

검색어: "무선이어버드 시장 동향"

- 결과: 두루뭉술한 개요 기사 몇 개
- 가격, 기술, 경쟁사 같은 세부는 빠짐

사람 애널리스트는 큰 주제를 만나면 나눠서 조사합니다. "기술 트렌드는 따로, 가격대는 따로, 주요 기업은 따로 찾아보자." 이걸 코드로 옮긴 게 하위 질의 분해입니다.

[큰 주제] 무선이어버드 시장 동향 → LLM이 하위 질의로 분해:

1. 무선이어버드 ANC 노이즈캔슬링 트렌드
2. 무선이어버드 가격대 동향 2026
3. 무선이어버드 주요 제조사 점유율

→ 각각 검색 → **[종합]** 세 결과를 하나의 리포트로 합침

2. 구조화 출력으로 하위 질의 받기

LLM에게 "질의를 나눠 줘"라고 하면 자유 텍스트로 답해 파싱이 흔들립니다. 8강처럼 **Pydantic** 구조화 출력으로 리스트를 강제합니다.

```
pip install -U pydantic # 이미 8강에서 설치했다면 생략
```

```
# 실행: python "sections/22_research_agent/06_복잡한-주제를-하위질의로-쪼개기.py"
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat

from pydantic import BaseModel, Field
from langchain_community.tools import DuckDuckGoSearchResults
from ch22_research import save_report # 리포트 저장은 기존 함수 재사용

class SubQueries(BaseModel):
    """[왜 구조화] LLM이 자유 텍스트로 질의를 내면 파싱이 흔들린다.
    Pydantic으로 '리스트'를 강제해 안정적으로 하위 질의를 받는다."""
    queries: list[str] = Field(description="원주제를 조사하기 위한 3~4개의 구체적 하위 검색 질의")
```

`with_structured_output(SubQueries)` 를 쓰면, LLM이 무슨 말을 하든 결과가 `queries: list[str]` 형태로 보장됩니다. 뒷단에서 `for q in queries` 로 바로 돌릴 수 있죠.

3. 주제 분해 함수

```
def split_topic(topic: str) -> list[str]:
    """큰 주제를 3~4개의 하위 질의로 분해한다(구조화 출력)."""
    llm = get_chat(temperature=0.2)
    structured = llm.with_structured_output(SubQueries)
    prompt = (
        f"너는 승승장구몰의 시장 조사 애널리스트다. 주제 '{topic}'를 깊이 조사하기 위해 "
        "서로 겹치지 않는 구체적 하위 검색 질의 3~4개로 쪼개라. "
        "각 질의는 한국어 검색어 형태로 작성하라."
    )
    try:
        return structured.invoke(prompt).queries[:4] # 최대 4개로 제한
    except Exception as e:
        # 구조화 실패 시 최소한의 폴백 질의
        print(f"[경고] 질의 분해 실패 → 기본 질의 사용: {e}")
        return [f"{topic} 시장 규모", f"{topic} 최신 동향", f"{topic} 주요 기업"]
```

짚어 둘 점

- `temperature=0.2` — 질의를 일관되게 나누려고 낮게.
- `"서로 겹치지 않는"` — 하위 질의가 비슷하면 같은 결과만 나오니, 다양성을 명시.
- `[:4]` — 너무 많이 쪼개면 검색 횟수·비용이 늘어 4개로 상한.
- 풀백 — 구조화마저 실패하면 기본 질의 3중으로라도 진행(03번 풀백 정신).

4. 하위 질의별 검색 (각각 풀백)

각 하위 질의를 따로 검색합니다. 질의 하나가 실패해도 전체가 멈추면 안 되니, 검색 단위로 풀백을 둡니다.

```
def search_one(query: str) -> str:
    """하위 질의 하나를 웹 검색해 결과 텍스트를 반환한다(실패 시 풀백)."""
    try:
        return DuckDuckGoSearchResults().invoke(query)
    except Exception as e:
        # [왜 풀백] 한 질의의 네트워크 실패로 전체 조사가 멈추면 안 된다.
        print(f"[경고] 검색 실패('{query}') → LLM 지식으로 대체: {e}")
        llm = get_chat(temperature=0.3)
        return llm.invoke(f"'{query}'에 대해 아는 범위에서 핵심 사실 3가지를 한국어로 정리하라.").content
```

질의별 풀백의 가치: 4개 중 1개만 검색이 막혀도, 나머지 3개는 정상 검색되고 막힌 1개만 LLM 지식으로 메웁니다. 부분 실패가 전체 실패로 번지지 않죠.

5. 결과 종합 — 하나의 리포트로

질의별 검색 결과를 모아 LLM에게 하나의 리포트로 합치게 합니다.

```
def synthesize(topic: str, findings: list[tuple]) -> str:
    """하위 질의별 검색 결과들을 하나의 마크다운 리포트로 종합한다."""
    llm = get_chat(temperature=0.3)
    # 각 결과를 1200자로 잘라 붙임(토큰 절약). findings = [(질의, 검색결과), ...]
    blocks = "\n\n".join(f"[하위질의] {q}\n{r[:1200]}" for q, r in findings)
    prompt = (
        f"주제: {topic}\n아래는 하위 질의별 검색 결과다. 이를 종합해 한국어 마크다운 리포트를 작성하라.\n"
        "'## 개요 / ## 핵심 동향 / ## 시사점' 섹션을 정확히 이 머리말로 포함하라.\n\n"
        f"{blocks}"
    )
    return llm.invoke(prompt).content
```

r[:1200] — 검색 결과를 1200 자로 자릅니다. 결과가 길면 토큰이 폭증하니, 핵심만 넣어 비용·속도를 관리합니다(토큰 관리). 4 개 질의 × 1200 자면 약 5 천 자, 한 번에 종합하기 적당하죠.

6. 전체 파이프라인

분해 → 검색 → 종합을 한 함수로 엮습니다.

```
def deep_research(topic: str) -> str:
    """주제 분해 → 하위 검색 → 종합의 전체 파이프라인."""
    queries = split_topic(topic)          # 1) 분해
    print(f"[분해] {len(queries)}개 하위 질의:")
    for i, q in enumerate(queries):
        print(f"  [{i}] {q}")
    findings = []
    for i, q in enumerate(queries):       # 2) 질의별 검색
        print(f"[검색 {i}] {q} ...")
        findings.append((q, search_one(q)))
    return synthesize(topic, findings)    # 3) 종합

if __name__ == "__main__":
    topic = "경쟁 무선이어버드 시장 동향"
    print("=" * 60)
    print(f"[조사 시작] {topic}")
    body = deep_research(topic)
    path = save_report(topic, body)       # 저장은 기존 함수 재사용
    print("=" * 60)
    print(body[:600], "...")
    print("=" * 60)
    print("[저장 완료]", path)
```

예상 출력

```
[조사 시작] 경쟁 무선이어버드 시장 동향
[분해] 4개 하위 질의:
  [0] 무선이어버드 ANC 노이즈캔슬링 기술 트렌드
  [1] 무선이어버드 가격대별 시장 동향 2026
  [2] 무선이어버드 주요 제조사 점유율
  [3] 무선이어버드 소비자 선호 기능 조사
[검색 0] 무선이어버드 ANC 노이즈캔슬링 기술 트렌드 ...
[검색 1] 무선이어버드 가격대별 시장 동향 2026 ...
[검색 2] 무선이어버드 주요 제조사 점유율 ...
```

[검색 3] 무선이어버드 소비자 선호 기능 조사 ...

개요

무선이어버드 시장은 ANC 보편화와 중저가 확산이 동시에 진행되며 ...

핵심 동향

- (ANC) 프리미엄 기능의 중저가 침투
- (가격) 5만원대 가성비 라인 급성장
- (점유율) 상위 3사 과점 속 중소 브랜드 약진

시사점

승승장구몰 PB는 5만원대 ANC 모델로 가성비 시장을 ... 공략 권장.

...

[저장 완료] /Users/.../reports/research_2026-06-07.md

관찰 포인트: 한 번 검색했을 때보다 각 측면(기술·가격·점유율·선호)이 골고루 담깁니다. 큰 주제를 나눠 조사한 효과죠.

7. 04번 방식과 비교

	04번 (에이전트 자율)	06번 (하위 질의 분해)
검색 통제	에이전트가 알아서	우리가 분해·검색 횟수 통제
깊이	주제에 따라 들쭉날쭉	측면별로 고르게 깊음
비용 예측	어려움(몇 번 검색할지 모름)	쉬움(질의 수 = 검색 수)
적합	가벼운 단발 조사	깊이 있는 정기 리포트

둘 다 쓸모가 있습니다. 가벼운 조사는 04번, 깊이가 필요한 정기 리포트는 06번이 어울리죠.

핵심 정리

- 큰 주제를 검색어 하나로 던지면 **결과가 얇음**.
- LLM 이 3~4 개 하위 질의로 분해(구조화 출력 with_structured_output) → 각각 검색 → 종합.
- **Plan-and-Execute** 를 리서치에 적용한 패턴.
- 질의별 폴백으로 **부분 실패가 전체 실패로 번지지 않게**.
- 검색 결과는 **1200 자로 잘라** 토큰·비용 관리.
- 04 번(자율)보다 **비용 예측·측면별 깊이**가 좋음.

검색 결과 교차 검증

검색 결과 **하나만 믿으면 위험**합니다. 잘못된 기사나 오래된 정보일 수 있죠. 하나의 핵심 주장을 서로 다른 검색어 2 개로 재검색해, 두 결과가 일치하는지 LLM 이 판단하고 **신뢰도(높음/보통/낮음)** 를 매깁니다. 단일 출처 맹신을 막는 안전장치입니다.

1. 단일 출처의 위험

검색 결과는 항상 옳지 않습니다.

- 광고성 · 과장 기사 ("올해 최고의 이어버드!")
- 오래된 정보 (2년 전 점유율을 최신처럼)
- 한 회사 입장만 반영 (자사 유리한 통계)

리포트에 "ANC가 시장을 주도한다"라고 썼는데, 그게 한 블로그 글 하나에서만 나온 주장이라면? 신뢰하기 어렵습니다. 사람 애널리스트는 중요한 주장일수록 여러 출처로 교차 확인합니다. 이걸 코드로 옮긴 게 교차 검증입니다.

주장: "무선이어버드 시장은 **ANC** 중심으로 성장한다" 를 두 검색어로 재검색합니다.

- 검색어 A: "무선이어버드 시장 성장 노이즈 캔슬링"
- 검색어 B: "TWS 이어버드 ANC 트렌드 2026"

→ 두 결과를 LLM이 비교 → [판정] 일치/부분일치/불일치 + 신뢰도(높음/보통/낮음)

2. 판정 결과를 구조화

교차 검증 결과를 일관된 형식으로 받으려고 Pydantic 스키마를 씁니다(06번과 같은 원리).

```
pip install -U pydantic # 이미 설치했다면 생략
```

```
# 실행: python "sections/22_research_agent/07_검색결과-교차검증.py"
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat

from pydantic import BaseModel, Field
from langchain_community.tools import DuckDuckGoSearchResults
```

```
class Verdict(BaseModel):
    """[왜 구조화] 교차검증 결론을 일관된 형식(일치여부 · 신뢰도 · 근거)으로 강제."""
    agree: str = Field(description="두 검색 결과의 관계: '일치' | '부분일치' | '불일치'")
    confidence: str = Field(description="주장 신뢰도: '높음' | '보통' | '낮음'")
    reason: str = Field(description="판단 근거 1~2문장(한국어)")
```

이렇게 하면 LLM 판정이 항상 **agree** (일치 여부) · **confidence** (신뢰도) · **reason** (근거) 세 칸으로 들어옵니다. 리포트에 "신뢰도: 보통" 같은 라벨을 자동으로 붙일 수 있죠.

3. 검색 함수 (폴백 포함)

```
def search(query: str) -> str:
    """검색어 하나로 웹 검색(실패 시 LLM 지식 폴백)."""
    try:
        return DuckDuckGoSearchResults().invoke(query)
    except Exception as e:
        print(f"[경고] 검색 실패('{query}') → LLM 지식으로 대체: {e}")
        llm = get_chat(temperature=0.3)
        return llm.invoke(f"'{query}'에 대해 아는 핵심 사실을 한국어로 3가지 정리하라.").content
```

06번과 같은 폴백 패턴입니다. 검색이 막혀도 LLM 지식으로 메워 교차 검증을 계속합니다.

4. 교차 검증 함수

핵심입니다. 같은 주장을 다른 검색어 2개로 본 뒤, LLM이 두 결과를 비교해 판정합니다.

```
def cross_check(claim: str, q1: str, q2: str) -> Verdict:
    """동일 주장을 서로 다른 검색어 2개로 검증해 일치/신뢰도를 LLM이 판단한다."""
    # [핵심] 같은 사실을 '다른 검색어'로 두 번 본다 → 한 출처에만 의존하지 않는 교차검증.
    r1, r2 = search(q1), search(q2)
    llm = get_chat(temperature=0) # [왜 0] 판정은 일관돼야 하니 결정적으로
    structured = llm.with_structured_output(Verdict)
    prompt = (
        f"검증할 주장: \"{claim}\"\\n\\n"
        f"[검색A: {q1}]\\n{r1[:1200]}\\n\\n"
        f"[검색B: {q2}]\\n{r2[:1200]}\\n\\n"
        "두 검색 결과가 이 주장을 지지하는지 비교하라. 서로 어긋나거나 근거가 약하면 신뢰도를 낮춰라."
    )
    try:
        return structured.invoke(prompt)
    except Exception as e:
```



```
print(f"[경고] 판단 실패 → 보수적 결론: {e}")
# [왜 보수적] 검증이 실패하면 '믿을 수 없음'으로 처리하는 게 안전하다.
return Verdict(agree="불일치", confidence="낮음", reason="검증 과정 오류로 신뢰 불가")
```

설계 포인트

- `temperature=0` — 판정은 일관성이 생명이라 결정적으로(분류·검증은 항상 0, 3·16강).
- `r1[:1200]` — 각 검색 결과를 잘라 토큰 관리(06번과 동일).
- 보수적 폴백 — 판정이 실패하면 "불일치/낮음"으로 처리합니다. 검증이 안 됐는데 "믿을 만함"으로 두는 건 위험하니, 의심스러우면 신뢰도를 낮추는 쪽이 안전하죠.

5. 실행

```
if __name__ == "__main__":
    claim = "최근 무선이어버드 시장은 노이즈 캔슬링(ANC) 기능 중심으로 성장하고 있다."
    q1 = "무선이어버드 시장 성장 노이즈 캔슬링"
    q2 = "TWS 이어버드 ANC 트렌드 2026"

    print("=" * 60)
    print("[검증 대상 주장]", claim)
    print(f"[검색어A] {q1}")
    print(f"[검색어B] {q2}")
    print("=" * 60)
    v = cross_check(claim, q1, q2)
    print("[교차검증 결과]")
    print(f"  일치여부 : {v.agree}")
    print(f"  신뢰도   : {v.confidence}")
    print(f"  근거     : {v.reason}")
```

예상 출력

```
=====
[검증 대상 주장] 최근 무선이어버드 시장은 노이즈 캔슬링(ANC) 기능 중심으로 성장하고 있다.
[검색어A] 무선이어버드 시장 성장 노이즈 캔슬링
[검색어B] TWS 이어버드 ANC 트렌드 2026
=====
[교차검증 결과]
  일치여부 : 일치
  신뢰도   : 높음
  근거     : 두 검색 모두 ANC 보편화와 프리미엄 기능 확산을 시장 성장 요인으로 언급함.
```

두 검색어가 서로 다른데도 같은 결론을 가리키면 신뢰도가 "높음"으로 올라갑니다. 반대로 한쪽은 ANC, 다른 쪽은 "가격이 핵심"이라고 하면 "부분일치/보통"이나 "불일치/낮음"이 나오죠.

6. 리포트에 신뢰도 붙이기

교차 검증을 리서치 파이프라인(06번)과 결합하면, 리포트의 각 핵심 주장에 신뢰도 라벨을 달 수 있습니다.

```
## 핵심 동향
- ANC 중심 성장 ..... [신뢰도: 높음 ✅]
- 5만원대 가성비 급성장 ..... [신뢰도: 보통 ⚠️]
- 점유율 상위 3사 과점 ..... [신뢰도: 낮음 ❓ 출처 부족]
```

이렇게 하면 리포트를 읽는 사람이 "어디까지 믿어야 하는지" 를 한눈에 압니다. "신뢰도 낮음" 항목은 사람이 추가 확인하면 되죠. 16강(환각 억제)의 정신 — 불확실하면 불확실하다고 표시 — 을 리서치에 적용한 겁니다.

7. 교차 검증의 비용과 트레이드오프

교차 검증은 공짜가 아닙니다.

```
- 검색이 2배 (주장당 검색어 2개)
- LLM 판정 호출 추가
→ 모든 문장을 검증하면 비용 · 시간 폭증
```

그래서 실무에서는 중요한 핵심 주장 몇 개만 교차 검증합니다. "이 리포트의 결론을 좌우하는 주장"만 골라 검증하고, 부수적 내용은 그냥 두는 식이죠. 비용과 신뢰도 사이의 균형을 잡는 게 핵심입니다.

핵심 정리

- 검색 결과 하나만 믿으면 위험(광고·구식·편향).
- 같은 주장을 다른 검색어 2 개로 재검색 → LLM 이 비교 → 신뢰도 판정.
- 판정은 Verdict(일치여부·신뢰도·근거)로 구조화, temperature=0 으로 일관되게.
- 검증 실패 시 보수적으로(불일치/낮음) — 의심스러우면 신뢰도를 낮춤.
- 리포트 주장에 신뢰도 라벨을 달면 독자가 믿을 범위를 읽(16 강 정신).
- 비용이 2 배 들므로 핵심 주장만 선별해 검증.